MLDS 490 Homework 1

Student: Shiva (Northwestern MSAI)

GitHub username: `Sbboss`

Repository: `mlds490-hw1`

Link: https://github.com/Sbboss/mlds490-hw1

Part 1 is the shuttle MDP, Part 2 is REINFORCE on CartPole and Pong, and Part 3 repeats both games with a baseline. Every number below comes from a file in the repo: `results/` for the training runs and evaluations, and `shuttle/validation_results.md` and `validation_pg_results.md` for the checks. Test names like T-R2 or T-P6 are the entries in those two files (R is returns and the estimator, C is CartPole, P is Pong, B is baselines). The headline CartPole figures use seed 0, but the five-seed tables say more about how well each method works.

How to run

```
git clone https://github.com/Sbboss/mlds490-hw1 && cd mlds490-hw1
python3 -m venv .venv && source .venv/bin/activate
pip install -U pip && pip install torch && pip install -r requirements.txt
```

Part 1 (CPU on the local Mac; the measured re-run was 2.4 s including pytest; 29 pytest checks plus 38 validation checks):

```
cd shuttle
python run_shuttle.py && python validation.py && python -m pytest -q test_shuttle.py
cd ..
```

Policy-gradient validation (CPU; the Pong block needs ale-py and takes a few minutes):

```
python -m validation.run_validation
python -m validation.run_validation --pong
python -m validation.test_tb5          # gradient-variance experiment, after C0 seed 0 exists
```

CartPole (CPU, one seed, 2000 episodes, a few minutes):

```
python -m pg.train_cartpole --baseline none --seed 0 --episodes 2000 \
    --out results/cartpole/C0_none/seed0
python -m pg.evaluate --env cartpole --run results/cartpole/C0_none/seed0
python -m pg.plot_results
```

The full matrix (C0 to C5, plus the two variants C7 and C8) is `scripts/run_cartpole_all.sh`, or the same flags in a local loop.

Pong runs on deepdish with exactly one GPU. Torch numbers the cards differently from `nvidia-smi`: torch device 0 is the 48 GB RTX PRO 5000, which `nvidia-smi` lists as index 1. Every Pong job sets `CUDA_VISIBLE_DEVICES=0`, and `nvidia-smi` showed memory in use on that card only.

```
ssh <netid>@mlds-deepdish5.ads.northwestern.edu
cd ~/mlds490_hw1 && source ~/venvs/mlds490/bin/activate     # setup: scripts/remote_setup.sh
export CUDA_VISIBLE_DEVICES=0
export OMP_NUM_THREADS=1
nice -n 10 python -m pg.train_pong --baseline none --lr 0.01 --workers 16 \
    --episodes_per_update 16 --max_episodes 25000 --max_hours 4 --seed 0 \
    --out results/pong/P0_none/seed0
python -m pg.evaluate --env pong --run results/pong/P0_none/seed0 --workers 16
```

Part 3 uses the same command with `--baseline value` (learned state-value) or `--baseline ema`. `tmux` isn't installed on that node, so long jobs run under `nohup setsid` and write `log.txt`. `bash scripts/sync_down.sh` pulls the results back.

The extended runs sit next to the command above and don't replace it. They use lr 0.001, switch the moving-average early stop off, and stop at 3 hours. Both were started together, 16 workers each, on the same 48 GB card.

```
env CUDA_VISIBLE_DEVICES=0 OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 \
  nice -n 10 python -m pg.train_pong --baseline none --lr 0.001 \
    --workers 16 --episodes_per_update 16 --max_episodes 1000000 \
    --max_hours 3 --early_stop 0 --seed 0 \
    --out results/pong/ext_P0_lr1e-3/seed0
env CUDA_VISIBLE_DEVICES=0 OMP_NUM_THREADS=1 MKL_NUM_THREADS=1 \
  nice -n 10 python -m pg.train_pong --baseline value --lr 0.001 \
    --workers 16 --episodes_per_update 16 --max_episodes 1000000 \
    --max_hours 3 --early_stop 0 --seed 0 \
    --out results/pong/ext_P1_lr1e-3/seed0
python -m pg.evaluate --env pong --run results/pong/ext_P0_lr1e-3/seed0 \
    --n 500 --seed 10000 --workers 16
python -m pg.evaluate --env pong --run results/pong/ext_P1_lr1e-3/seed0 \
    --n 500 --seed 10000 --workers 16
```

Hardware and runtimes. Part 1 and the shuttle tests ran on the local Mac (the measured re-run was 2.4 s including pytest). The policy-gradient validation suite, including `--pong`, also ran on the local Mac. The CartPole matrix ran on the local Mac on CPU (`run_meta.json` has no CUDA device; C0 seed 0 finished in 23.0 s); C0 to C5 with up to 6 seeds at a time took about 4 minutes wall-clock. Pong ran on deepdish (`mlds-deepdish5`): every Pong `run_meta.json` records `device: cuda`, one RTX PRO 5000 Blackwell (48 GB), 16 actor processes per job, at most 32 across jobs, `nice -n 10`. The remote stack is in `results/env_versions.txt`: Python 3.13.5, torch 2.11.0+cu128, numpy 2.5.4, gymnasium 1.4.0, ale-py 0.12.1, CUDA driver 615.71 / CUDA 13.4. One 2000-episode Pong learning-rate run took between 148 and 272 s, depending on how long the rallies got. A 500-episode CartPole evaluation on the Mac takes a few seconds when the policy fails right away and about a minute when it reaches the 500-step cap.

Part 1: Shuttle dispatch

Model and assumptions

`s` is the number of customers waiting, `s = 0, …, 200`. Each period I either hold or dispatch. A dispatch costs `cf = 100` and takes up to `K = 15` customers. After the decision, every customer still waiting costs `ch = 2`, charged before the new arrivals. Arrivals are `A ~ Uniform{1,…,5}`, so the next state is `min(200, leftover + A)`. Arrivals beyond 200 are dropped for free. Costs are minimized, `γ = 0.95`, and everything runs in float64. If dispatch and hold differ by at most `1e−9`, the code holds, so the policy dispatches only when dispatch is strictly better.

"Enumeration" here means backward induction over all `T = 500` stages. A literal tree of `2^500` action sequences can't be built. The one test that does enumerate literally uses 12 steps with deterministic arrivals (`2^12 = 4096` sequences), and it matches the DP to `1e−13`.

Results

quantity	measured
`V_0(0)` at `T = 500`, and `V*(0)`	576.6702
`V*(15)`	676.6702 (= `V*(0) + 100`)
`V*(200)`	3821.586
optimal policy, all three methods	dispatch iff `s ≥ 13`
value-iteration iterations (`ε = 1e−10`)	580
policy-iteration iterations (start: always hold)	6
largest gap between `V_0` (`T = 500`) and `V*` from value iteration	4.86e-9
Monte Carlo of the optimal policy from `s = 0`	576.56 ± 0.16 (SE)
average cost per period `g`	32.34
dispatch frequency	0.2133

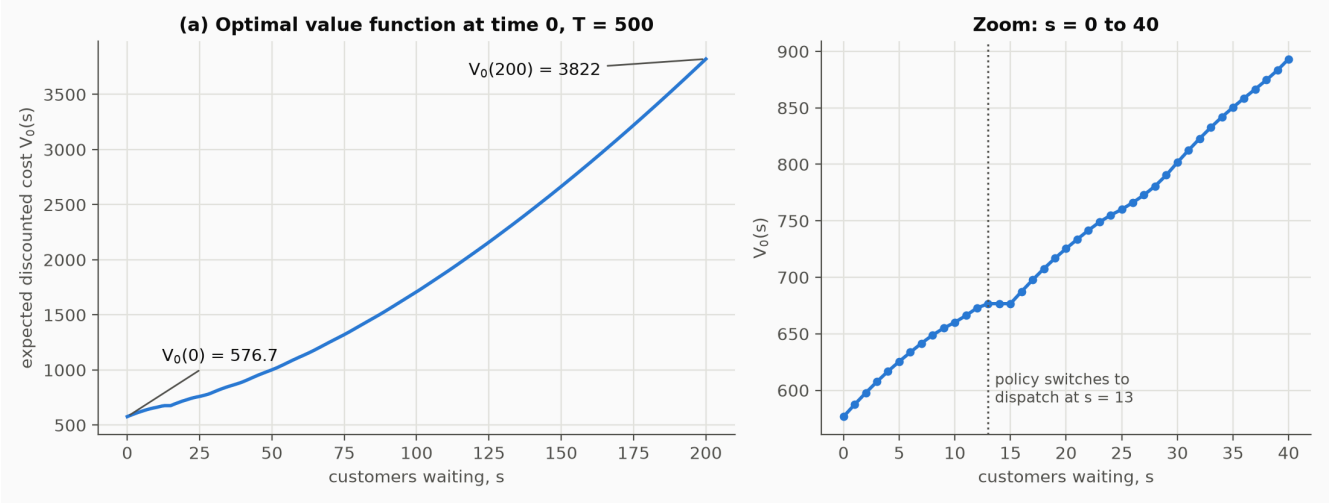


Figure 1. Optimal value at time 0 against customers waiting, from exhaustive backward induction over $T = 500$. It agrees with the infinite-horizon value to 5e-9, so the horizon is long enough to count as the steady problem.

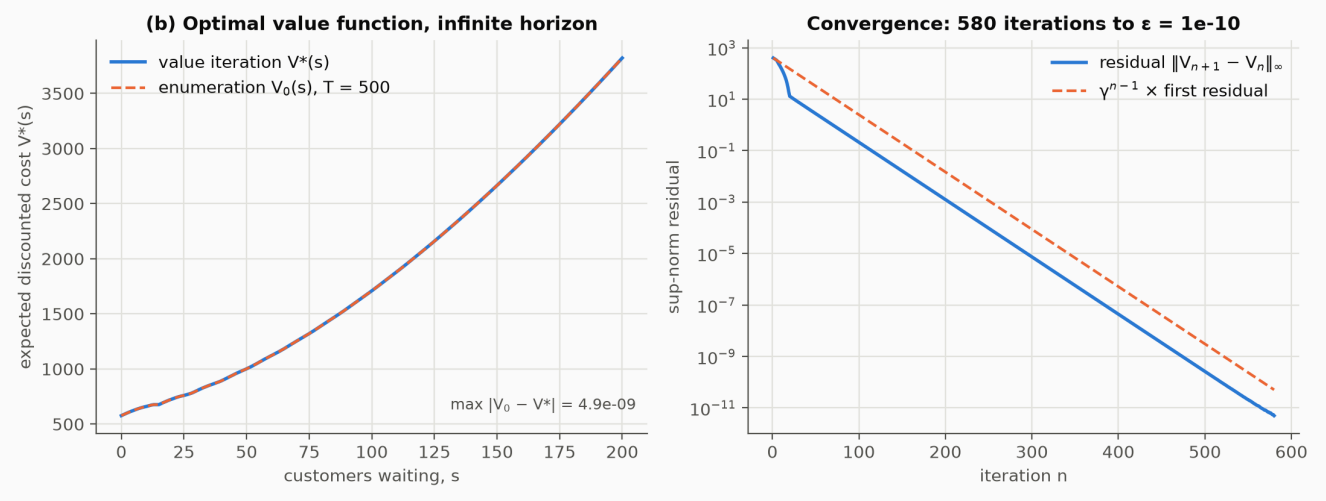


Figure 2. V∗ from value iteration. The flat stretch on `s = 13, 14, 15` is the dispatch region. All three states pay 100 and leave an empty queue, so they share the same continuation.

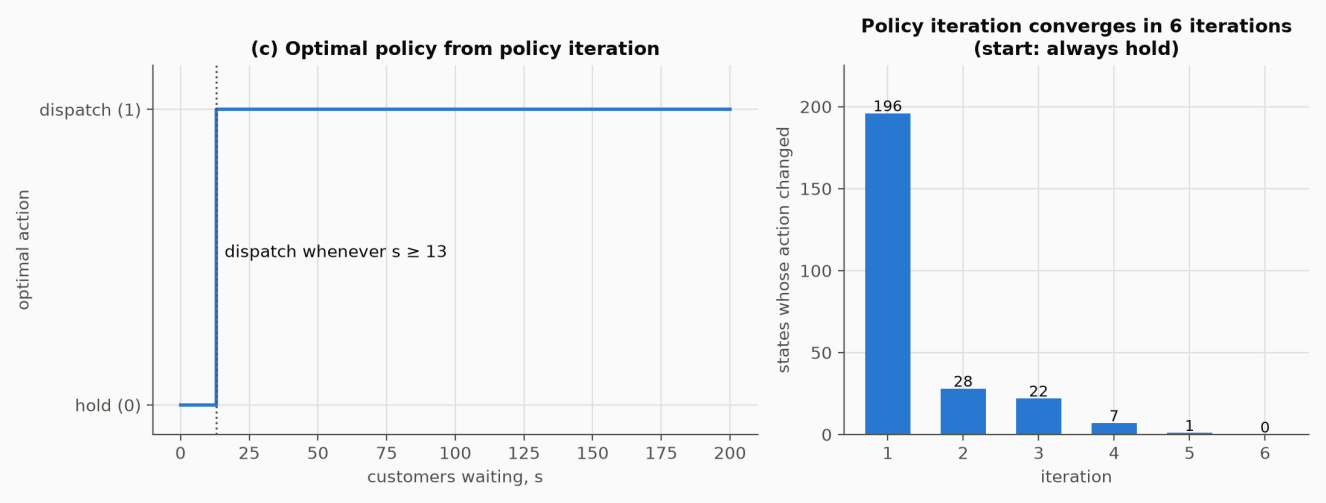


Figure 3. Optimal action against customers waiting, and the number of states whose action changes at each policy-iteration step. The threshold is 13. Policy iteration from "always hold" finishes in 6 iterations (196, 28, 22, 7, 1, 0 states changed).

Edge cases

I looked for edges that come from this particular cost structure, not generic empty-input checks.

- **The cap.** Caps 30, 200, 300 and 400 give the same value and the same policy on the states they share (largest difference under `1e−12`). Cap 10 doesn't. Overflow customers vanish for free, so the problem gets artificially cheap: `V(0) = 356.04` instead of 576.67, and the optimal policy never dispatches. That is a modeling artifact. The test `cap_10_artifact` checks both the value and the empty dispatch set.
- **When the holding cost is charged.** Charging `ch` on the post-arrival queue instead of the post-decision queue adds a constant. Measured `V(0)` goes from 576.67 to 696.67, a difference of

120, and the threshold stays at 13. A constant that doesn't depend on the action can't change the policy, so the ambiguity in the problem statement doesn't matter for the decision.

- **Ties.** With `cf = 0`, dispatch and hold tie at `s = 0` and the code holds. The test `limit_cf_zero` requires dispatch for every `s ≥ 1` and hold at 0.
- **Boundary transitions.** Holding at 198 puts mass 0.2 on 199 and 0.8 on 200. Holding at 199 or 200 puts mass 1 on 200. Dispatching from 200 goes to `{186,…,190}`. All of these are checked exactly.
- **Convexity.** I expected `V` to be convex. It isn't: the minimum second difference is -3.778. A minimum over actions of functions doesn't have to stay convex, so I shouldn't have assumed it. This is the one Part 1 prediction that failed.

How I checked it

Re-run before submission: 38 checks, 0 exact failures, 1 prediction not held (convexity), and 29 pytest tests passed. The full table is `shuttle/validation_results.md`. These are the checks that pin down the algorithm, as opposed to the file format:

- One- and two-stage values match a hand calculation (`1e−13`).
- The 12-step deterministic enumeration matches backward induction (`1e−13`).
- Value iteration and policy iteration agree to `8e−11` and return the same policy. Policy iteration reaches that policy from always-hold, from always-dispatch and from a random start.
- Policy iteration is monotone: costs never increase from one iteration to the next.
- A Monte Carlo run of the optimal policy from `s = 0` gives 576.56 ± 0.16 against `V* = 576.67` (`z = −0.68`).
- The optimal policy beats "never dispatch" (`V(0) = 2203.9`) and "always dispatch" (`V(0) = 2000`).
- With `ch = 0` the value is 0 and the policy never dispatches. With `γ = 0` it never dispatches either, because one dispatch saves at most `ch · 15 = 30`, which is less than 100.
- The slopes `dV/ds` at `s = 50, 100, 199` are 10.6, 16.9 and 24.9, all below `ch / (1 − γ) = 40`.

Discussion

The threshold is 13 because a dispatch costs 100 and clears the queue, while holding costs `2s` now and leaves a bigger queue for later. Below 13, holding plus the discounted future still costs less than paying 100. States 13, 14 and 15 all dispatch, and dispatching from any of them leaves an empty queue, so `V` is flat across them. `T = 500` is close enough to infinite: `γ^500` is about `7e−12`, and the measured gap to the infinite-horizon value is `4.9e−9`.

Part 2: REINFORCE

Estimator

The loss is averaged over the episodes in a batch and summed over time inside an episode:

```
L = −(1/B) Σ_episodes Σ_t  w_t · log π(a_t | s_t) · (G_t − b_t)
```

G_t is the reward-to-go, computed backward: $G_t = r_t + \gamma\, G_{t+1}$. Part 2 uses $b_t = 0$.

I use $w_t = 1$ in every primary run (`--gamma_weight off`). The exact gradient of $\mathbb{E}[\sum_t \gamma^t r_t]$ multiplies the step-t term by γ^t. At $\gamma = 0.95$ that factor is about 0.006 by step 100, so the late part of a long CartPole episode would barely move the weights. The unweighted version is the usual basic REINFORCE. It optimizes a slightly different objective, because it reweights the state distribution. T-R2 shows that the two gradients differ, and config C3 turns the γ^t weight on.

On Pong, the return resets whenever a point is scored. A point is $r = \pm 1$ and ends a rally, and without the reset a later point leaks credit into earlier rallies. With rewards $[0, 0, -1, 0, 0, 1]$ and $\gamma = 0.99$, the reset gives $G[2] = -1$, and without it $G[2] \approx -0.030$. The flag is `--pong_reset_at_point` and it is on by default.

No primary run standardizes returns. Standardizing acts as a baseline, so it would make Part 2 and Part 3 hard to compare. The flag exists, and any run that uses it is labeled.

The last policy layer starts at 0.01 times the default weights with bias 0, so the initial policy is close to uniform (entropy $\ln 2 \approx 0.693$). Actions are drawn with `torch.multinomial(..., generator=)`, because `Categorical.sample` in this torch build doesn't take a generator. That is what lets a seed replay.

The optimizer is Adam. Its first update moves every parameter with a nonzero gradient by exactly the learning rate (measured: ± 0.010000, and 0 where the gradient is zero). A baseline that shrinks the gradient therefore doesn't take a bigger Adam step. Any gain shows up as a less noisy curve.

CartPole-v1

$\gamma = 0.95$, Adam learning rate 0.01 as specified, one episode per update, an MLP $4 \to 128 \to 2$, 2000 episodes, five seeds (0 to 4). The return is plain Monte Carlo. There is no bootstrap at the 500-step truncation, so the estimator is unbiased for the truncated-horizon objective and V near the cap comes out slightly low. I left that limitation in place. The environment is `CartPole-v1` with default settings, and each episode records whether it ended because $|x| > 2.4$, because $|\theta| > 12°$, or because of the time limit.

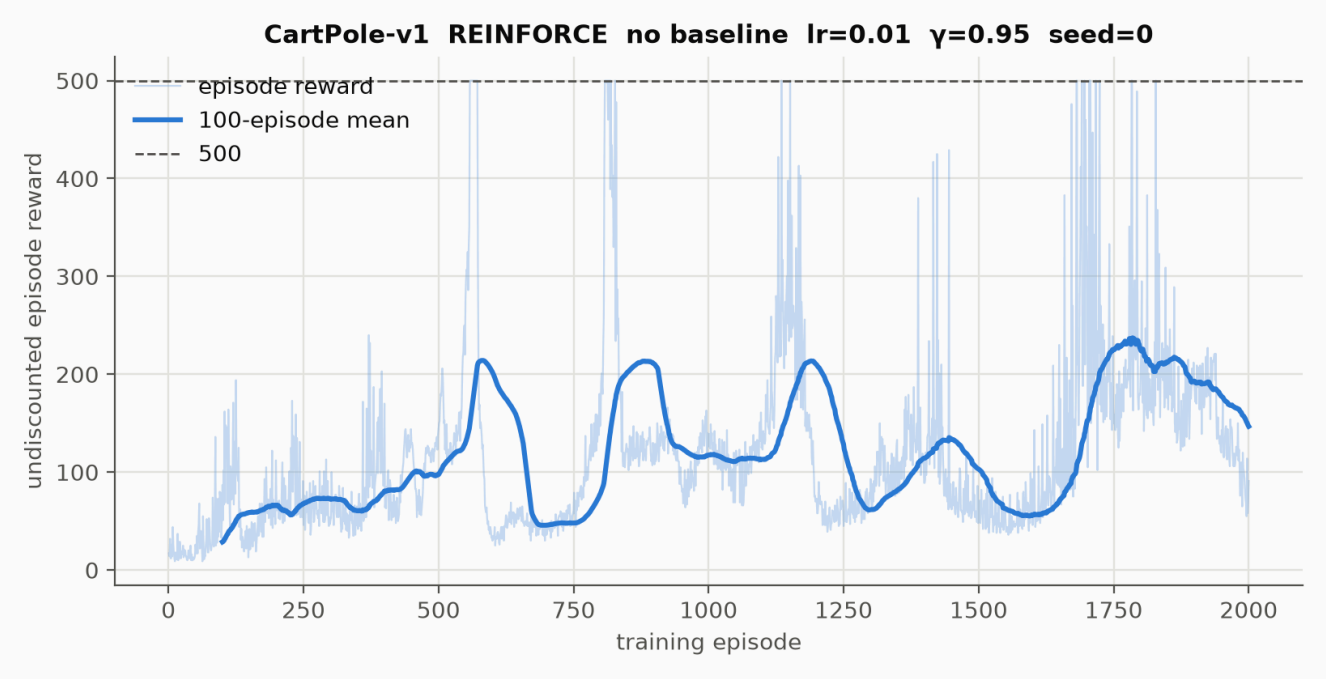


Figure 4. CartPole, no baseline, seed 0, lr 0.01, $\gamma = 0.95$. Thin line: undiscounted episode return. Bold line: trailing 100-episode mean, drawn only from episode 100. Dashed line: the 500-step cap. Seed 0's moving average ends at 147, well above the random-policy mean of about 22 and well short of the cap.

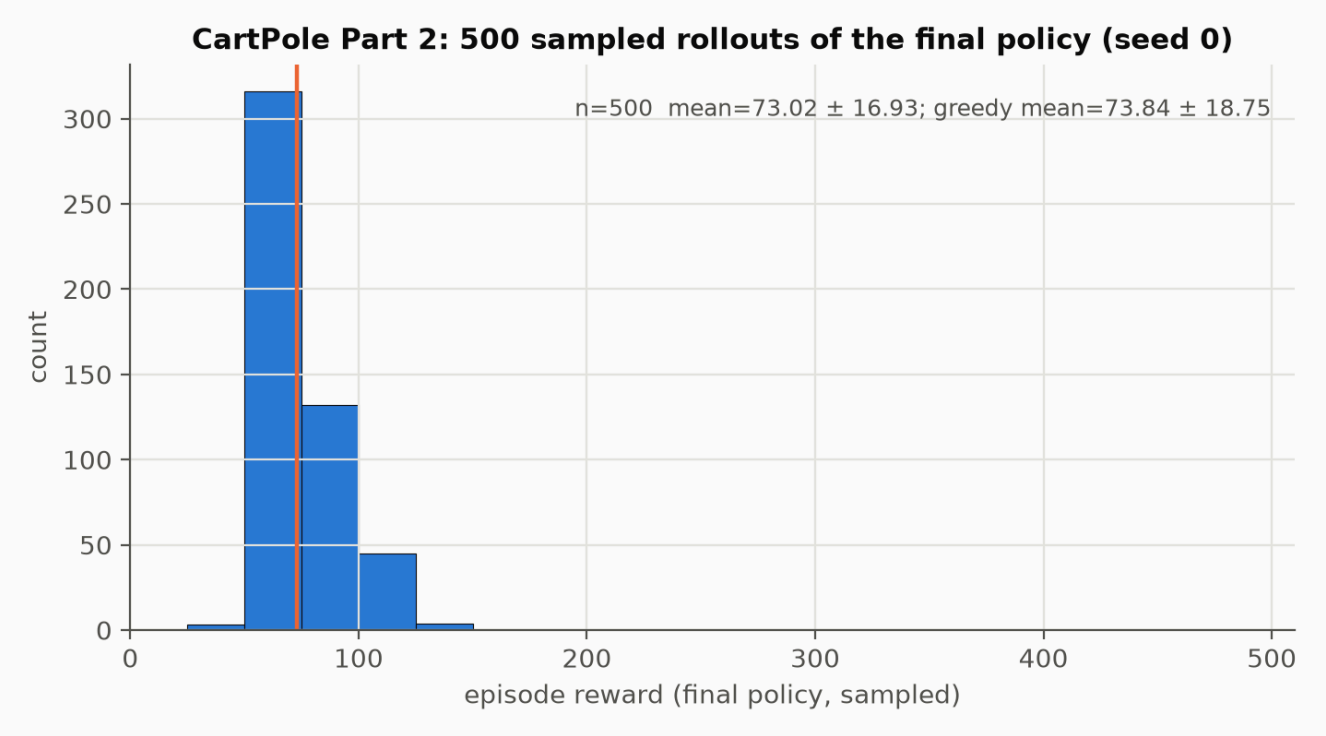


Figure 5. 500 sampled rollouts of the final seed-0 policy (evaluation seeds start at 10000, not the training seed). Mean 73.0, standard deviation 16.9, $n = 500$. Greedy rollouts: 73.8 ± 18.8. There is no spike at 500, because this final policy never survives the time limit. All 500 episodes end by the angle limit.

The training moving average and the final-policy evaluation are different numbers, and I report the evaluation. Seed 0's training MA100 is 147, while the 500-episode evaluation of the saved final weights is 73. Five checkpoints from the last 20% of that run, 50 episodes each, scored 60, 267, 196, 199 and 77, a spread of more than 100. The final save is one draw from a noisy late policy. That is why I fixed the rule before looking at results: evaluate the final weights and publish the spread.

seed	training MA100	500-ep mean ± std (sampled)	greedy mean ± std	how episodes ended
0	146.8	73.0 ± 16.9	73.8 ± 18.8	angle 100%
1	9.3	9.3 ± 0.7	9.4 ± 0.7	angle 100%
2	181.0	293.5 ± 85.7	340.7 ± 108.0	x-limit 34%, angle 57%, time-limit 9%
3	92.4	127.8 ± 12.0	128.6 ± 12.8	angle 100%
4	9.3	9.3 ± 0.7	9.4 ± 0.7	angle 100%

Across the five seeds the training MA100 averages 88, below the 100 I had set as the point where I'd start trying variants. Two seeds collapsed to episodes of about 9 steps, which is worse than the random policy (about 22). The lr 0.01, one-episode-per-update run is still the primary Part 2 result. The variants are labeled and don't replace it:

- **C7**, 8 episodes per update, lr 0.01, seeds 0 to 2. Final training MA100: 351, 192, 139. None collapsed during training. The saved final policies evaluate at 182 ± 22, 165 ± 10 and 157 ± 9, and nearly every episode ends with the cart leaving the track. The training curve overstates the policy that got saved.
- **C8**, lr 0.003, one episode per update, seeds 0 to 2. Final MA100: 146, 152, 143. Final-policy evaluations: 133 ± 10, 153 ± 9, 185 ± 27. None reached the cap.
- **C4**, lr 0.001, seeds 0 to 2. Final MA100: 485, 221, 132. Final-policy evaluations: 476 ± 53, 195 ± 28, 149 ± 22. Seed 0 is the only one of the three that mostly hits the time limit (68% sampled, 99% greedy). The smaller step is more reliable than 0.01 on this problem, and one seed still never reaches the cap.

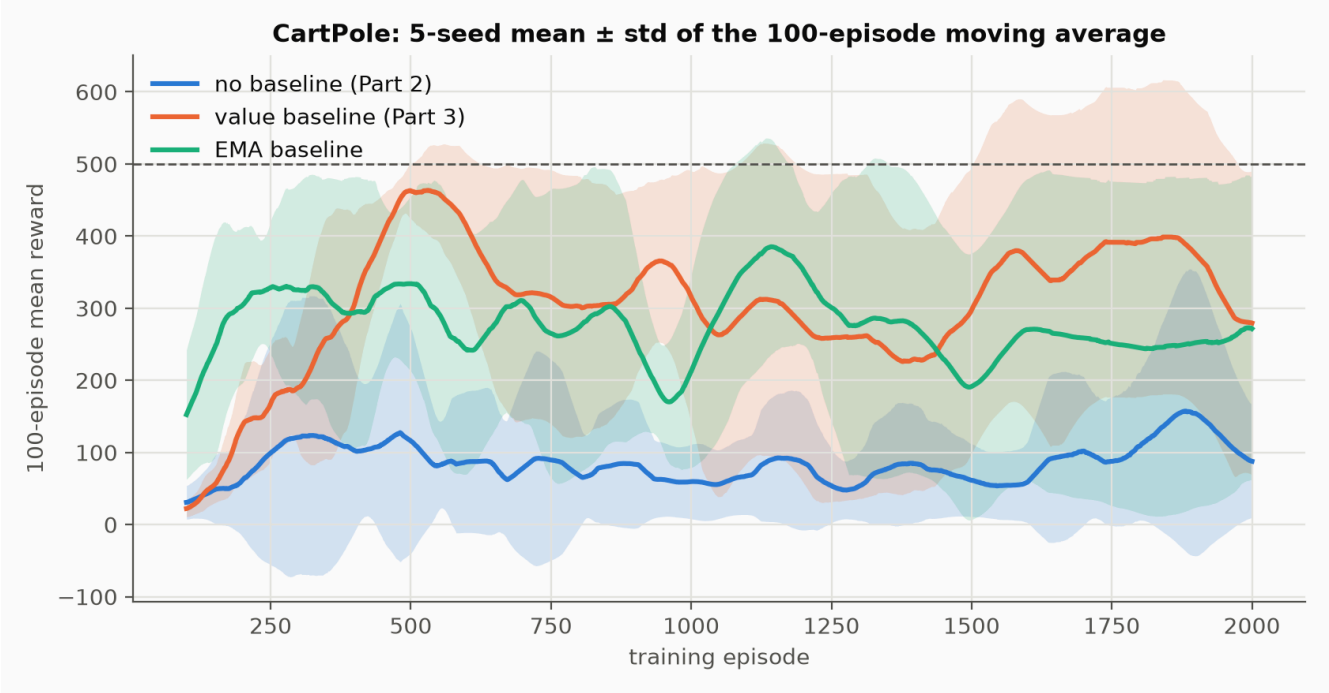


Figure 6. Mean ± one standard deviation, across five seeds, of the 100-episode moving average. The no-baseline mean stays lower (88 ± 78). The value and EMA bands are wider (280 ± 211 and 272 ± 209), because those runs include both a collapse to length 9 and runs that sit on the cap. The band crosses the dashed line at 500 because the standard deviation does, and a single episode can't score above 500.

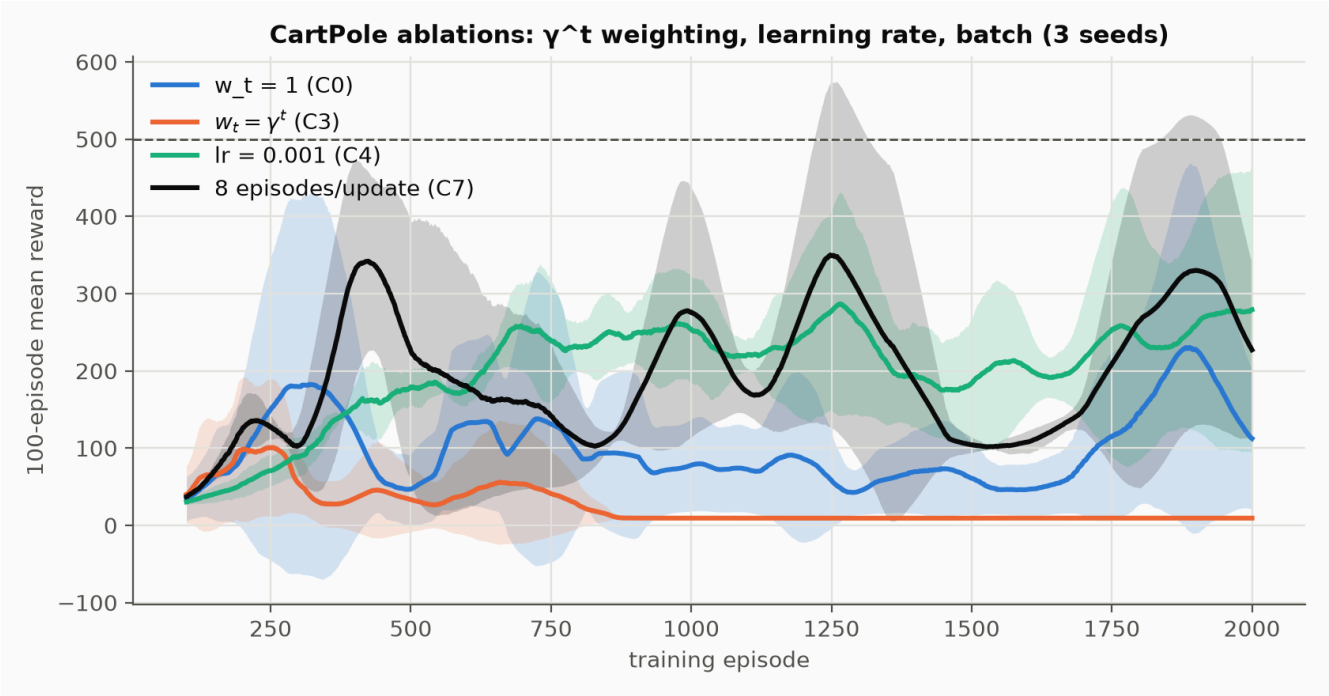


Figure 7. Three-seed ablations. `w_t = γ^t` (C3) doesn't learn: every seed ends at MA100 ≈ 9.3. lr 0.001 (C4) and 8 episodes per update (C7) both beat the primary lr 0.01 curve.

The γ^t weight removed the learning signal. In C3 (three seeds, 2000 episodes, lr 0.01) the final MA100 is 9.3, 9.3 and 9.5, and the 500-episode evaluations sit on that floor. T-R2 predicts this. At γ

= 0.95 the weight at step 100 is already negligible, and CartPole's most useful gradient often arrives late, when the pole is actually falling.

Negating the reward makes the agent fail fast. In C5 (three seeds, 300 episodes, rewards multiplied by -1) the mean length of the last 50 episodes is 9.3, 9.2 and 9.4, all under the ceiling of 15 I predicted (a random policy lasts about 22). The sign of the advantage is doing its job.

γ = 0 didn't stay at the random policy. With γ = 0 every G_t is 1 and the score function has mean zero, so the expected gradient is zero, and I expected three seeds of 1000 episodes to stay near length 22 (T-C8). They didn't. The last-100 mean lengths are 9.3, 9.5 and 9.4, and the final entropy is about 0.001. Each episode is one noisy sample of a mean-zero gradient, and Adam at lr 0.01 still takes steps of about 0.01 whatever the gradient's size. A thousand of those steps pushed the policy onto a short angle failure. The expected gradient is zero, but the optimizer on one-episode batches doesn't see the expectation. I didn't run the optional sweep over γ ∈ {0.5, 0.8, 0.99}.

Termination causes. I guessed that γ = 0.95 gives a short horizon and that some failures would come from the cart drifting out (|x| > 2.4). On the headline seed that's wrong: all 500 evaluation episodes end by angle. On seed 2, which did learn, 34% end by cart position, 57% by angle and 9% hit the time limit. The drift only shows up once the policy is good enough to survive the early angle failures. The seed-0 histogram is unimodal around 70, not bimodal, and has no mass at 500.

Pong-v5

The environment is ALE/Pong-v5 with the defaults: frameskip 4, sticky-action probability 0.25, six actions. The network chooses between index 0 and 1, which are sent to the emulator as actions 2 (RIGHT) and 3 (LEFT). The state is the difference of two preprocessed 80×80 frames, stored as int8 in {-1, 0, 1}. The first difference in an episode is all zeros, because there is no previous frame. preprocess is the function from the assignment, applied to a copy. The original slices are views and would erase the caller's frame, and T-P3 checks that the input is unchanged.

Actors are a spawn pool. Each worker builds its own environment, sets CUDA_VISIBLE_DEVICES="", and plays one full episode with a frozen weight vector. I don't use Gymnasium's vector environments, because their auto-reset can splice the next episode onto the return. The learner recomputes log π on the GPU with the same weights, so the update is on-policy. T-P6 measured the gap between the actor's log-probability and the learner's recomputation at 0 on a 945-step episode.

The architecture is the Karpathy-style MLP, 6400 → 200 → 2, with a batch of 16 episodes all played with the same weights. Sixteen is also the worker count, since the pool size is min(workers, episodes per update) and a batch of 16 can't feed 32 actors. The benchmark's fastest setting was 32 workers (14372 env-steps/s). 24 workers reached only 88.5% of that, outside a 10% band, but 32 workers would need a batch of at least 32. The batch stays at 16. Two jobs at 16 workers each use the 32-worker cap and no more.

Learning-rate sweep (no baseline, 2000 episodes, seed 0, batch 16). I fixed the selection metric beforehand: the final training MA100.

lr	final MA100	entropy at the end	rally length near the end
0.01	**−0.98**	0.044	~3400
0.001	−17.27	0.45	~1900
0.0003	−20.09	0.69	~1100
0.0001	−20.40	0.69	~1000

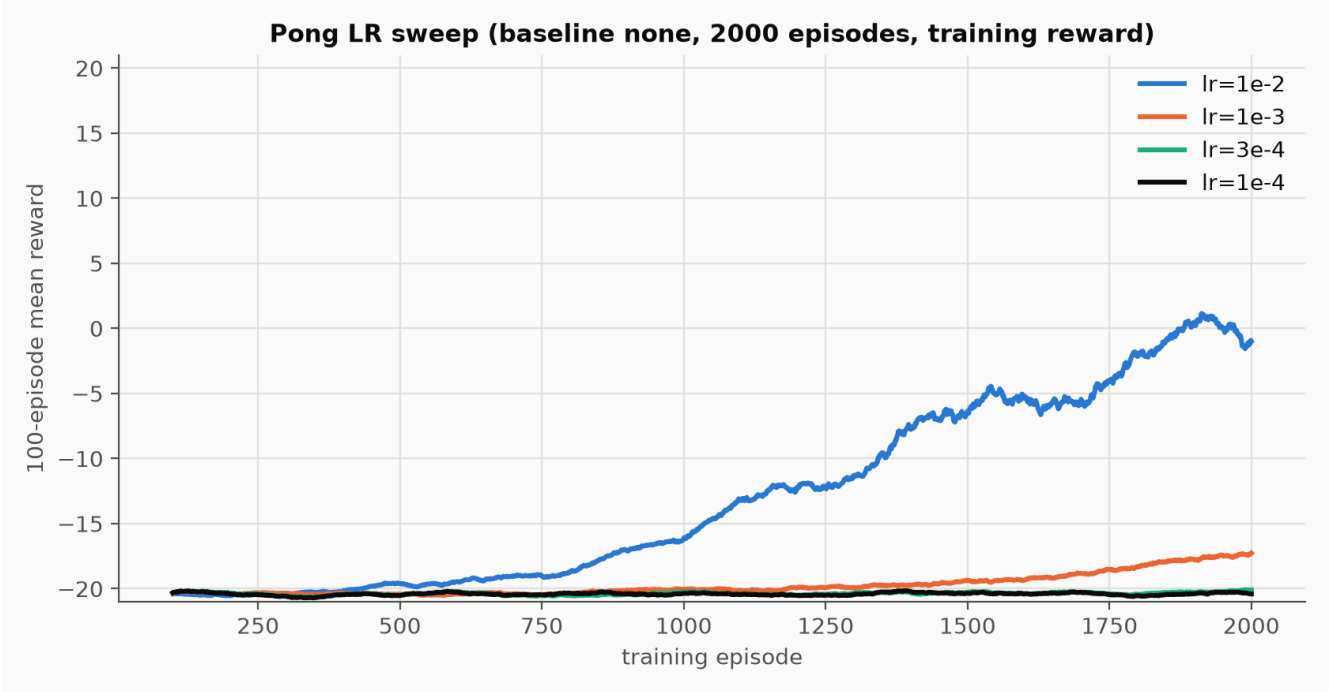


Figure 8. 100-episode mean of the training reward for the four learning rates, each stopped at 2000 episodes. Only lr = 0.01 leaves the random-agent floor inside that window, and it's also the rate the assignment asked for, so the primary full runs keep it. The curve at 0.01 reaches about +0.8 and ends at −1.0. lr 0.001 is still near −17 here. That ranking doesn't survive the 3-hour runs below, so the sweep horizon was too short.

A random agent scores about −20.7, and the standard error of a 100-episode mean is about 0.1, so a rise of 0.3 is the smallest change worth calling learning. At lr 0.01 the average rises by about 19 points and the rallies get three times longer. The policy also gets very peaked (entropy 0.04, against a maximum of 0.69). The fraction of index-0 actions stays near 0.45, so it hasn't collapsed onto one button, only onto a confident choice in each state. The two smallest rates never leave entropy 0.69 and never leave −20. lr 0.001 sits in between, with MA100 at −17.3, entropy still 0.45 and rallies of about 1900.

A 200-episode smoke test at lr 0.001 ran before the sweep. It stayed finite, kept entropy at 0.693 and kept the action fraction at 0.50. That only shows the loop isn't diverging, not that it learns. Inside the 2000-episode window lr 0.01 is the rate that learns, which is why the primary runs stay at 0.01. The extended runs later in this section show that the window was too short for 0.001.

The full Part 2 run is seed 0, lr 0.01, batch 16, no baseline and no return standardization. It stopped at episode 5232, after 986 seconds, because the 100-episode mean had moved only from −2.29 to

−2.08 over 3000 episodes. That rule compares two endpoints. Inside the window the curve still swung: the moving average peaked at +2.65 at episode 3092 and finished at −2.08.

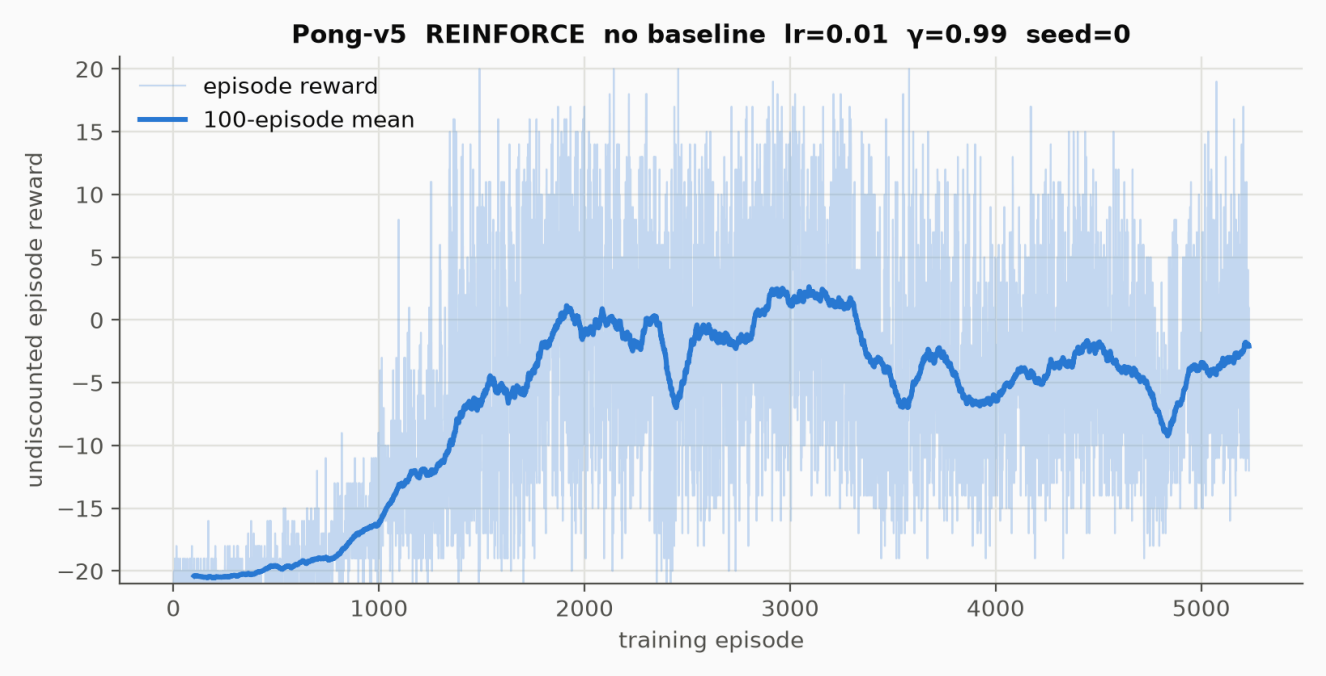


Figure 9. Pong, no baseline, seed 0, lr 0.01, γ = 0.99. The moving average leaves the random-agent floor near −21, spends a few thousand episodes around zero, and is stopped at −2.1. Individual episodes still cover almost the whole score range.

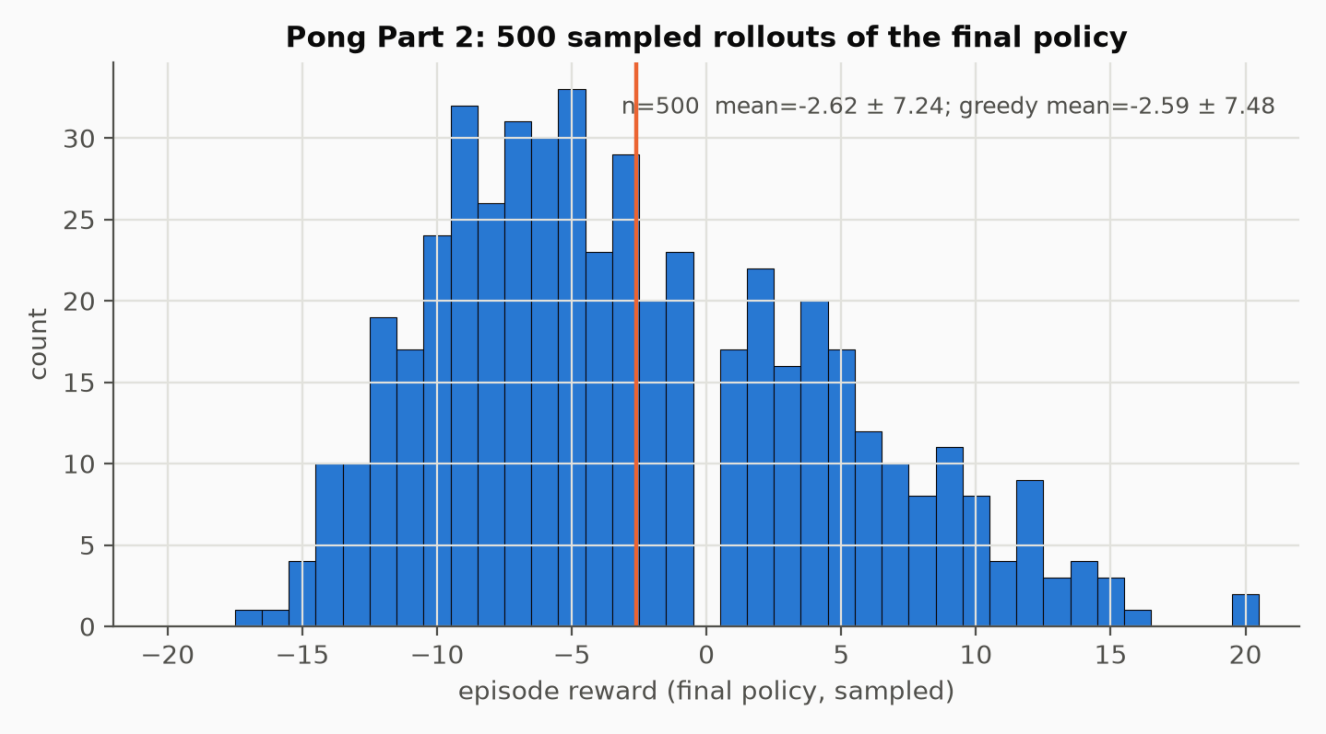


Figure 10. 500 sampled rollouts of the final Part 2 policy (evaluation seeds from 10000). Mean −2.62, standard deviation 7.24, n = 500, range −17 to +20. Greedy rollouts: −2.59 ± 7.48, range −21 to +20. The mass sits a

little below zero. A few games reach +20, but this isn't a solved player.

Five checkpoints from the last 20% of training, 50 episodes each, scored −3.80, −3.68, −3.04, −3.16 and −3.78, which agree to about one point. The final-policy lottery that showed up on CartPole doesn't show up here. Entropy at the end of training was 0.023, so the policy is confident, and the fraction of index-0 actions stayed near 0.45, so it didn't lock onto one button.

Extended run (lr 0.001, no baseline)

This isn't the Part 2 result, which is the lr 0.01 run above. I chose lr 0.001 after that run and the 2000-episode sweep were finished. The moving-average early stop was off and the episode ceiling was 1,000,000, so the run ended when `run_meta.json` recorded 10801.9 seconds (3.00 hours). It played 42,128 episodes, and the entropy at the last update was 0.055.

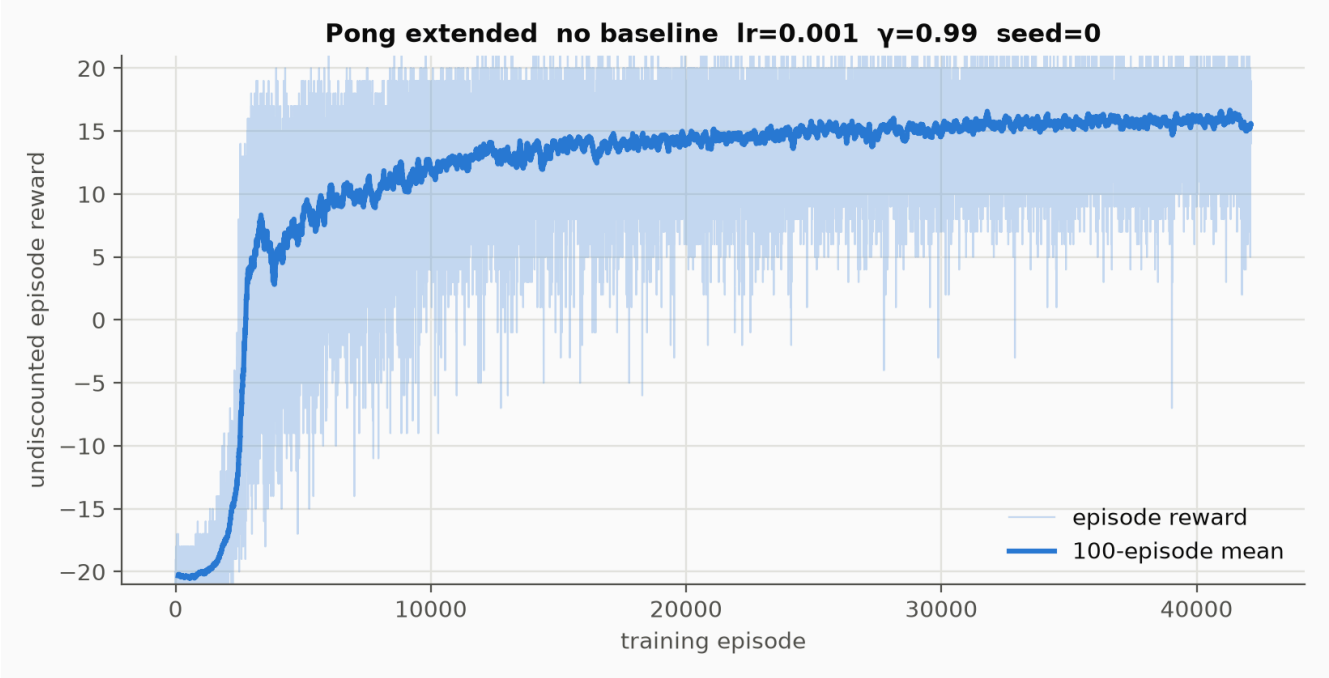


Figure 11. Extended run, no baseline, seed 0, lr 0.001, $\gamma = 0.99$, early stop off. The 100-episode mean leaves −21 later than the lr 0.01 curve and is still near +15.6 when the 3-hour clock stops it. The primary lr 0.01 run had already stopped at −2.08.

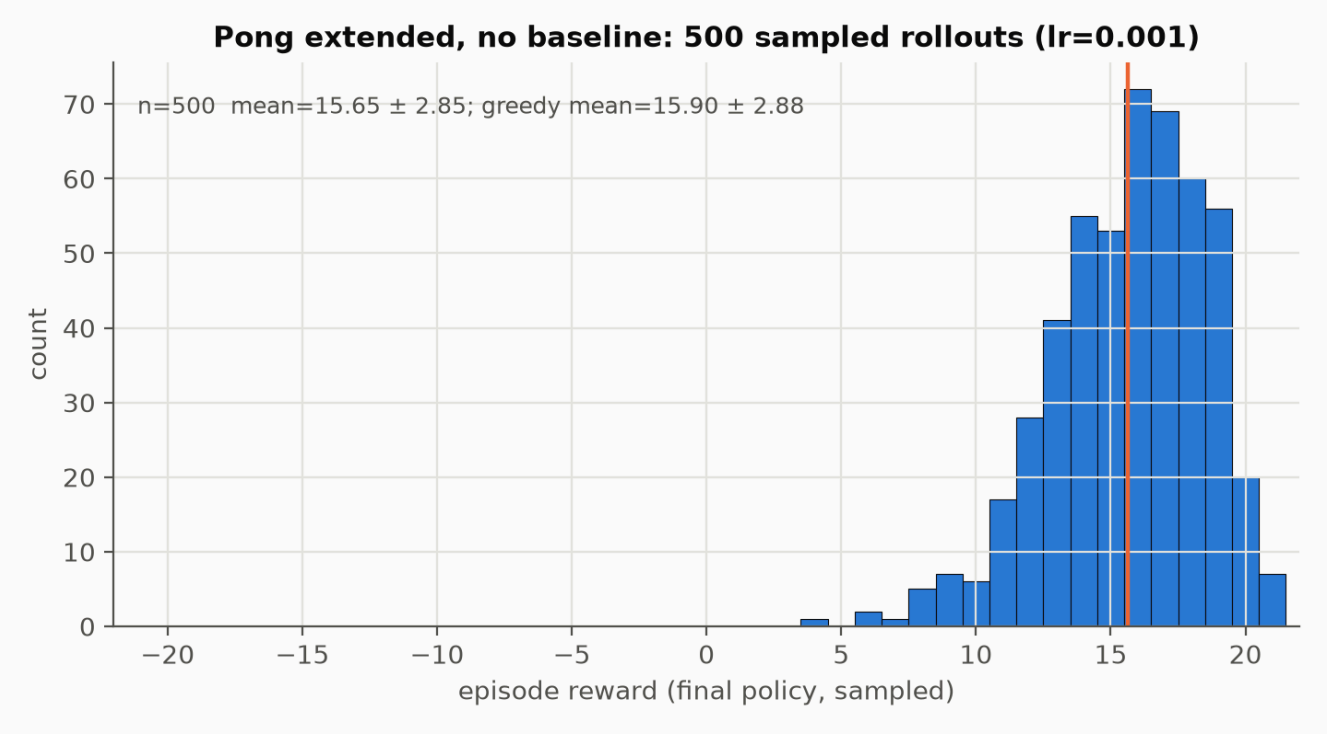


Figure 12. 500 sampled rollouts of that final policy, evaluation seed 10000, the same seed as the primary runs. Mean 15.65, standard deviation 2.85, n = 500, range 4 to 21. Greedy: 15.90 ± 2.88. Five checkpoints from the last 20% of training, 50 episodes each, scored 16.5, 16.1, 16.2, 16.4 and 15.3.

The 2000-episode sweep favored lr 0.01. At that horizon lr 0.001 sat at −17.27. After about 40,000 episodes the order reversed: this run ends at a moving average of +15.56 and a 500-episode mean of +15.65, against −2.08 and −2.62 for the primary no-baseline run. The sweep horizon was too short.

The two runs also didn't get the same amount of training. This one ran for 3.00 hours, while the primary no-baseline run's `run_meta.json` clock is 987 seconds, about 16 minutes (0.27 hours). The comparison mixes learning rate with training length. The four-run table, with the value-baseline pair, is in the Part 3 extended-run section.

How I checked the estimator and the Pong pipeline

Exact checks that passed, with the measured error:

- **T-R1.** `[1,1,1]` at `γ = 0.5` gives `[1.75, 1.5, 1.0]`, and a forward-order bug would give `[1, 1.5, 1.75]`. The same rewards at `γ = 0.95` give `[2.8525, 1.95, 1.0]`. The Pong reset on `[0,0,-1,0,0,1]` matches the hand values to the printed precision, and the no-reset `G[2]` is −0.0297.
- **T-R2.** A 2-step, 3-state softmax MDP with 200,000 trajectories. With `w_t = γ^t`, every component of the sample mean is within 4 standard errors of the autograd gradient (largest `|z|` is 1.13). With `w_t = 1`, the sample matches the enumerated expectation of that estimator (largest `|z|` is 0.72) and differs from the true gradient by 147 standard errors in one component. At `γ = 0.5` the unweighted step-2 term is off by a factor of two.

- **T-R3.** A softmax bandit with `θ = 0` and rewards `(1, 0)`. The sample mean of the gradient at N = 200,000 is `(0.2509, −0.2509)`, against the closed form `(0.25, −0.25)`, with `|z| = 1.56`.
- **T-C4.** Two runs with the same seed produce the same first episode. Two fresh agents after three updates have bit-identical CPU weights. Leaving out `zero_grad` changes a parameter from 0.60 to 0.80, so the test can fail.
- **T-P1, T-P2, T-P3.** Over 1000 steps every env action is in `{2, 3}`. Forcing action 2 for 30 steps and then action 3 for 60 moves the right paddle 28 px one way and 78 px the other. `preprocess` doesn't modify its input. The first frame difference is identically zero. Later differences lie in `{−1, 0, 1}` and are nonzero on 96% of the steps after step 20 in the checked episode.
- **T-P6, T-P7, T-P10.** Learner and actor log-probabilities match to `0`. On a real episode, `G_t = r_t` at every scoring step (21 points checked). Killing a run after 3 episodes and resuming it lands on episode 5, with no duplicated CSV rows and with weights that aren't the initialization.

Predictions that did not hold:

- **T-P5.** I expected an untrained net to score within `[−21, −19]` on every one of 20 episodes. The rewards included a −18, because one sticky-action episode scored a point. The mean was −20.40, which is inside `[−21, −19.8]`. The usual description of a random agent ("about −20.7, every game in [−21, −19]") is a little too tight for 20 episodes. I still only call it learning when the 100-episode mean rises by more than about 0.3 and stays there. lr = 0.01 does that, and lr = 0.0001 doesn't (final MA100 −20.40).
- **T-P9, first attempt.** A scripted controller that chased the ball's current row scored −21 on every episode. The mask was zeroing every column from 64 on, which is where the ball is when the paddle has to hit it, and one frameskipped action moves the paddle about 11 rows, so the sign flipped every step. The replacement predicts the row at the paddle, reflecting off the walls, and holds still inside a 5-pixel band. On the deterministic environment it scores +9 on each of 10 episodes. The frames were fine and the controller wasn't. Training doesn't use this controller.

The first 10 CartPole steps for a fixed seed are below. The probabilities are 0.500 to three digits, which is the uniform initialization, and `G` falls by about `γ` each step because every reward is 1.

t	x	dx	theta	dtheta	P(a=0)	a	r	G	log π
0	0.027	−0.006	0.036	0.020	0.500	0	1	13.189	−0.693
1	0.027	−0.202	0.036	0.324	0.500	1	1	12.830	−0.694
2	0.023	−0.007	0.043	0.042	0.500	1	1	12.453	−0.694
3	0.023	0.187	0.044	−0.236	0.500	1	1	12.056	−0.693
4	0.027	0.382	0.039	−0.515	0.500	0	1	11.638	−0.693
5	0.034	0.186	0.029	−0.210	0.500	0	1	11.197	−0.693
6	0.038	−0.009	0.024	0.091	0.500	0	1	10.734	−0.692
7	0.038	−0.205	0.026	0.391	0.501	0	1	10.247	−0.692
8	0.034	−0.400	0.034	0.692	0.501	0	1	9.733	−0.692
9	0.026	−0.596	0.048	0.995	0.501	1	1	9.193	−0.695

The full row, including the raw arrays, is in `results/validation/cartpole_10step_trace.json`.

Part 3: Baselines

What I implemented

I used two baselines. Both are independent of the action, so neither changes the mean of the gradient.

1. **Primary: a learned state-value baseline.** A second network with the same trunk and one scalar output, trained with its own Adam at learning rate `1e−3` (both games). It regresses onto the same `G_t` the policy uses. The advantage is `G_t − V_φ(s_t)`, with `V` detached inside the loss. The value step comes after the policy step. On Pong the value pass is one epoch of minibatches of 4096. Advantages aren't standardized.
2. **Secondary: a constant EMA.** `b ← 0.9 b + 0.1 · mean(G)`, using only previous batches. The batch being updated still sees the old `b`, and the first batch sees `b = 0`. Using the current episode's own returns would make `b` depend on the actions just sampled.

CartPole

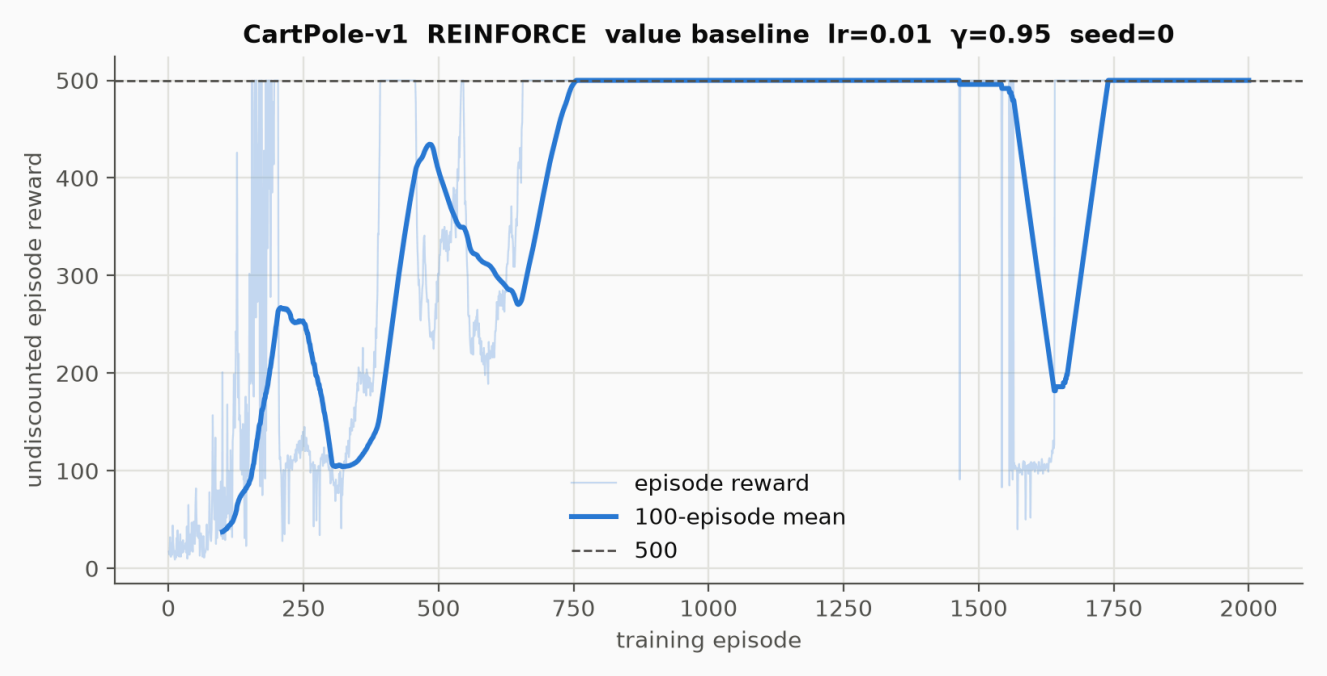


Figure 13. Value baseline, seed 0, same lr 0.01 and $\gamma = 0.95$ as Part 2. The moving average first reaches the cap near episode 744, falls to about 182 around episode 1640, and is back at 500 at the end. The histogram evaluates the final policy, not the dip.

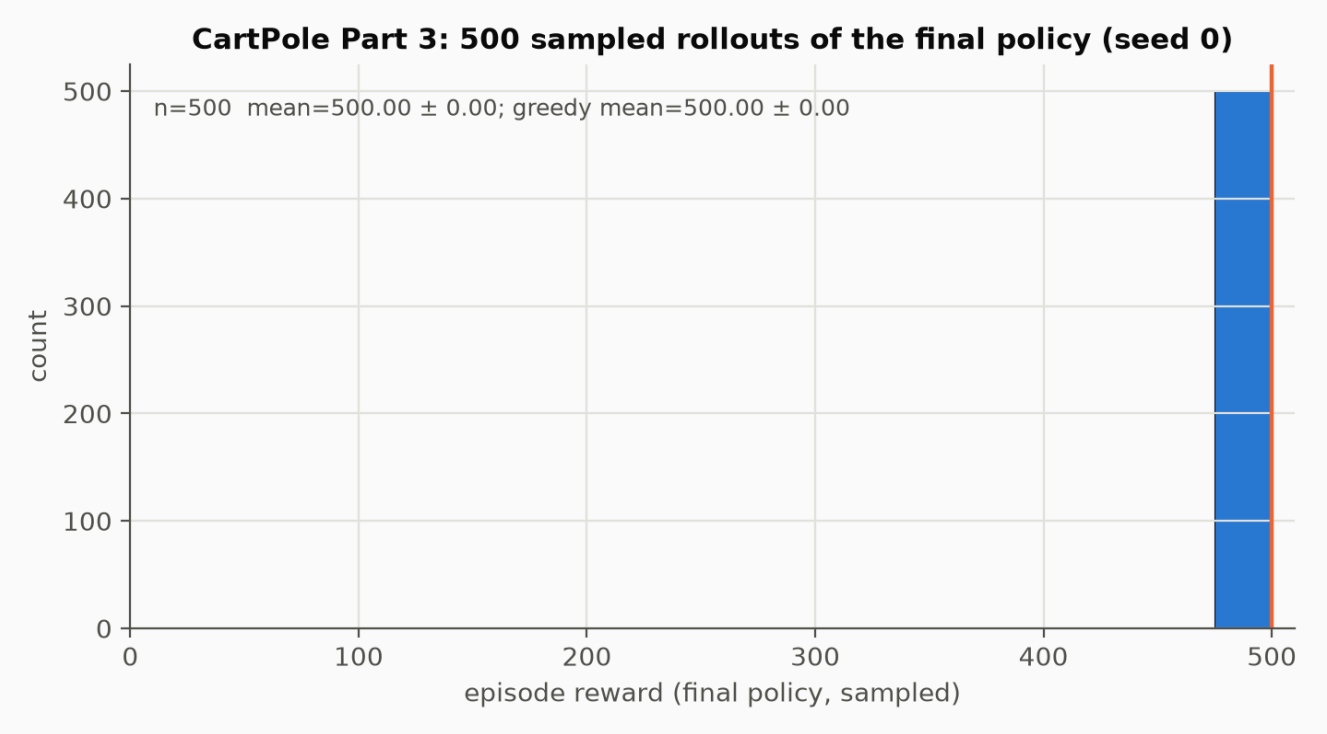


Figure 14. All 500 sampled rollouts of the final seed-0 value-baseline policy score 500 (std 0). Greedy rollouts are the same. Every episode ends at the time limit, which is the truncation spike the histogram is meant to show, and it's the whole distribution.

config	training MA100 by seed	500-ep sampled means
C0, no baseline	146.8, 9.3, 181.0, 92.4, 9.3	73.0, 9.3, 293.5, 127.8, 9.3
C1, value baseline	500, 161.2, 245.6, 9.3, 481.5	500, 27.6, 500, 9.3, 490.3
C2, EMA	469.7, 162.6, 9.5, 500, 217.2	452.7, 209.4, 9.3, 500.0, 96.8

Seed 0 is the one in the figure. Across seeds the value baseline is better on average, but it doesn't fix everything. Seed 3 collapses to length 9, the same failure as in Part 2, and seed 1 has a training MA100 of 161 while its final policy evaluates at 27.6. The EMA seed-0 evaluation is 452.7 ± 130.8 sampled and 491.9 ± 56.8 greedy, with 88% of episodes hitting the time limit. EMA seed 1 never hits the time limit: all 500 evaluation episodes end by cart position, with a mean of 209 ± 14. EMA seed 4 evaluates at 97 ± 16, all by angle.

The fair comparison uses the 500-episode means. Seed 0 gives 73 with no baseline and 500 with the value baseline. Across five seeds the means are 103 ± 118 (no baseline), 305 ± 262 (value) and 254 ± 216 (EMA). The value baseline didn't reduce the spread across seeds. Part 2 and Part 3 each have seeds that never leave the floor, so I can't say that a baseline tightens the seed distribution on these five seeds. These standard deviations are across the five seed means, not across the 500 episodes inside one seed.

The variance evidence (T-B5)

The policy was frozen at three checkpoints of C0 seed 0 (episode 200, episode 1000 and the final weights). For each checkpoint, 1000 fresh episodes were split into A and B, 500 each. A value network was fit on A, and the constant baseline was the mean return on A. On B, the same trajectories gave the policy gradient under `b = 0`, `b = const` and `b = V`.

A first 40-epoch fit left R^2 negative (about -0.05 to -0.30). That fit hadn't moved. Adam steps are about the size of the learning rate (T-R4), and `1e-3` for 40 steps can't lift a value head from 0 up to returns near 15. The reported fit keeps lr `1e-3` and runs until the training MSE flattens or 4000 epochs pass.

checkpoint	R^2 on B	tr(Cov) const / b=0	tr(Cov) V / b=0	cos(b=0, const)	cos(b=0, V)
episode 200	0.946	0.182	0.012	0.973	0.973
episode 1000	0.989	0.172	0.002	0.614	0.333
final	0.968	0.194	0.011	0.987	0.989

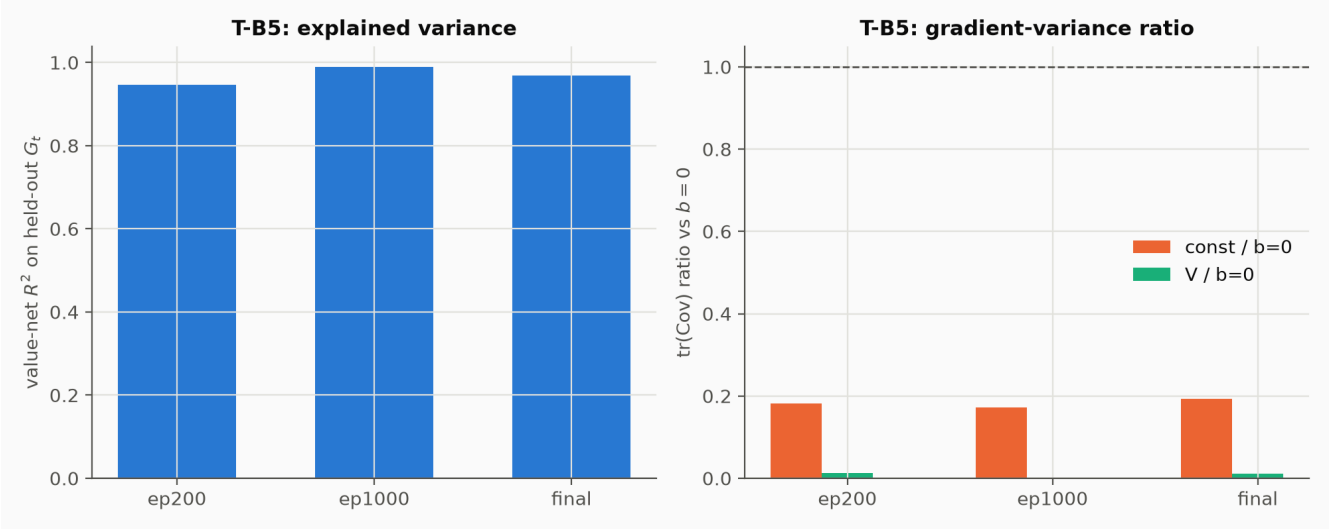


Figure 15. Left: once it's properly fit, the value network explains almost all of the held-out return variance. Right: both baselines cut the trace of the gradient covariance, and the state-value baseline cuts it by roughly 100×. The dashed line marks "no reduction."

The constant baseline gives about a 5× variance cut and stays aligned with the uncentered gradient except at the episode-1000 checkpoint, where the cosine falls to 0.61. The value baseline's cosine at that checkpoint is 0.33, not 1. The mean gradient there is small compared with the noise (the no-baseline trace is five times larger than at the other two checkpoints), so the cosine between two noisy means isn't a stable check of unbiasedness at that one checkpoint. At episode 200 and at the final weights the cosines are 0.97 and 0.99. The variance ratios are below 1 at every checkpoint, and R^2 is above 0.5 at every checkpoint, including the final one (0.97).

The toy-MDP version of the same claim (T-B1, N = 200,000, every reward shifted by +10) is exact. `b = 0`, `b = J` and `b = V^π` all match the true gradient within 1 standard error, and the covariance traces are 139.6, 3.58 and 0.11. For a 2-action bandit, the optimal constant baseline equals `J` when `π = (1/2, 1/2)`, weights the rarer action 9× when `π = (0.9, 0.1)`, and has the smallest variance of `{b*, J, 0}`.

Detaching works (T-B3). With the value head clamped at 0, the Part 3 gradient equals the Part 2 gradient exactly (largest absolute difference 0). After the policy backward pass, every value parameter has a null or zero gradient, and the value-loss backward pass doesn't change the policy gradient.

Pong with a baseline

The Part 3 Pong run uses the same lr 0.01 and batch of 16, plus the learned value baseline. Standardization stayed off, because Part 2 had already left −21 without it. About 10 minutes in, this run was holding 41 GB on the 48 GB card. It was stopped at episode 2752 and resumed from that checkpoint after the value update was changed to drop each minibatch. The 16 episodes that had been logged but not checkpointed were collected again. The resumed run stopped at episode 7488, because the moving average had moved only from 10.37 to 10.65 over 3000 episodes. The peak was +12.68 at episode 7099, and the final training MA100 is +10.65.

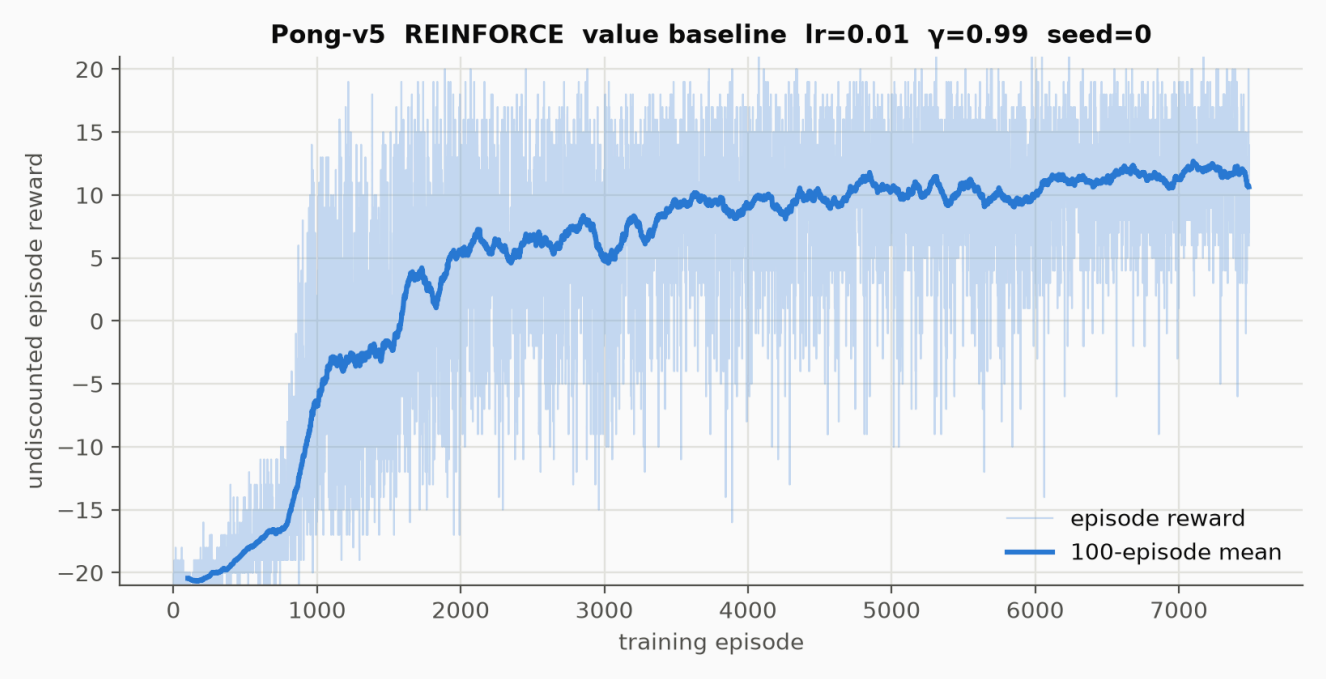


Figure 16. Pong, value baseline, seed 0, lr 0.01, $\gamma = 0.99$. The moving average crosses zero near episode 1500 and levels off near $+11$, about 13 points above the Part 2 run, which leveled off near -2.

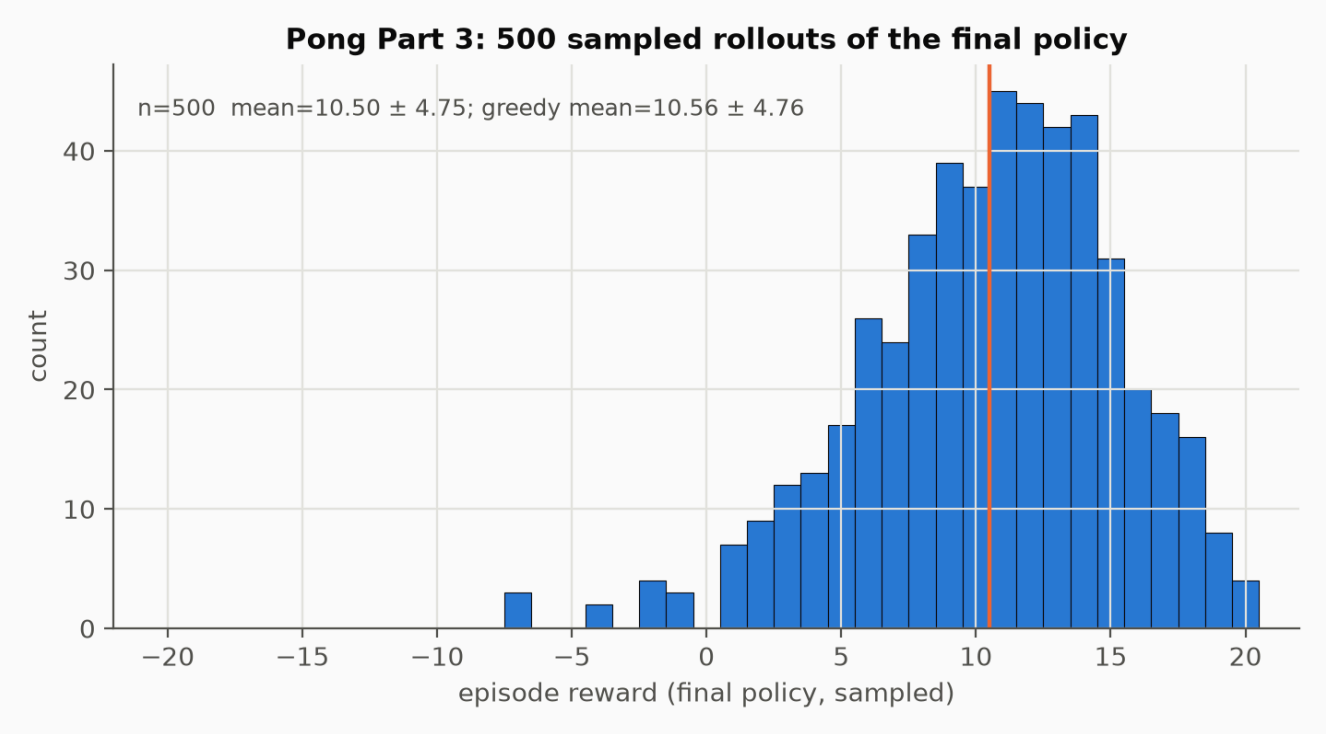


Figure 17. 500 sampled rollouts of the final value-baseline policy. Mean 10.50, standard deviation 4.75, $n = 500$, range -7 to $+20$. Greedy: 10.56 ± 4.76. Almost all of the mass is positive, and a handful of games still lose.

Five late checkpoints, 50 episodes each, scored 11.7, 10.7, 10.3, 10.9 and 11.4. As in Part 2, the late policies agree and the final save isn't a lucky outlier.

Extended run (lr 0.001, value baseline)

This has the same departure from the defaults as the extended Part 2 run: lr 0.001, early stop off, a 3-hour cap, seed 0 and 16 workers, run at the same time as that no-baseline job. I chose it after the primary results were in. `run_meta.json` records 10804.8 seconds and 39,232 episodes, and the last-update entropy was 0.056.

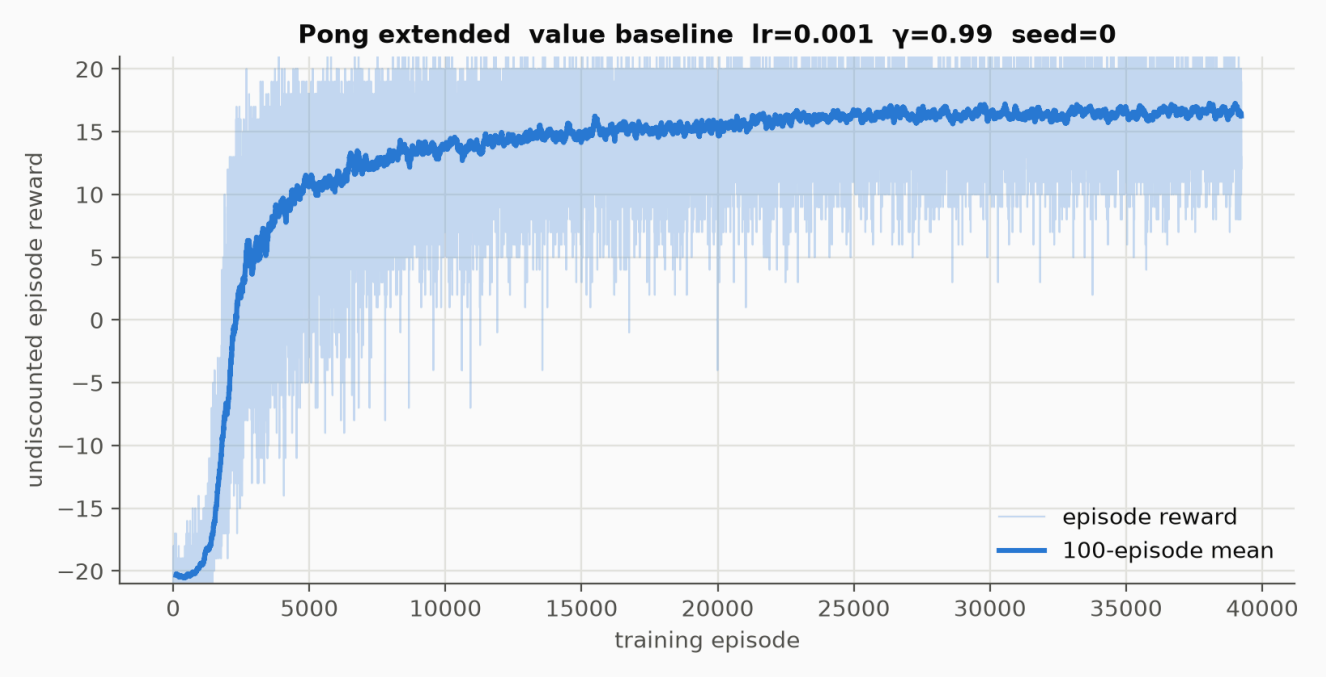


Figure 18. Extended run, value baseline, seed 0, lr 0.001. The moving average ends at +16.26. The primary value-baseline run, at lr 0.01, ended at +10.65.

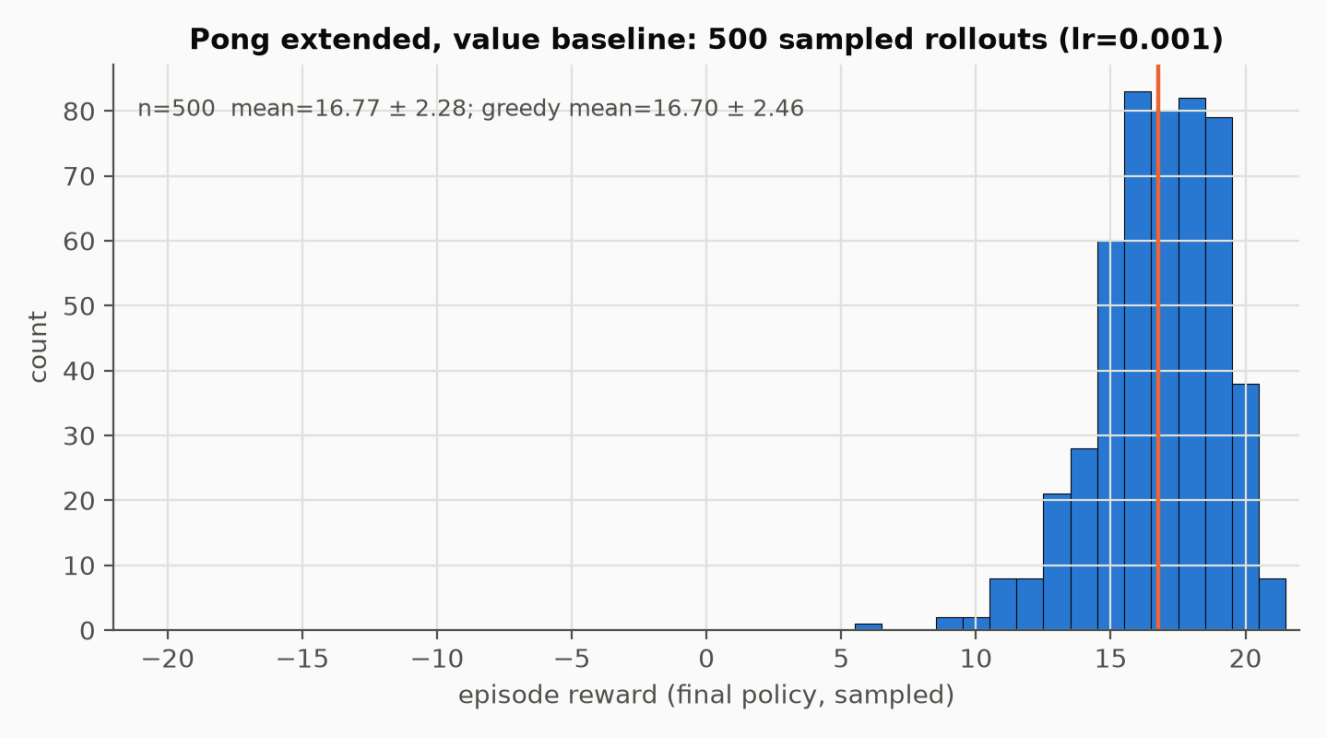


Figure 19. 500 sampled rollouts, evaluation seed 10000. Mean 16.77, standard deviation 2.28, n = 500, range 6 to 21. Greedy: 16.70 ± 2.46. Five late checkpoints, 50 episodes each, scored 16.8, 16.9, 17.2, 17.4 and 16.7.

run	episodes	wall-clock hours	final MA100	entropy at the end	500-ep sampled
lr 0.01, no baseline	5232	0.27	−2.08	0.023	−2.62 ± 7.24
lr 0.01, value baseline	7488	0.33	+10.65	0.020	+10.50 ± 4.75
lr 0.001, no baseline	42128	3.00	+15.56	0.055	+15.65 ± 2.85
lr 0.001, value baseline	39232	3.00	+16.26	0.056	+16.77 ± 2.28

Hours are `run_meta.json` seconds divided by 3600. The lr 0.01 value-baseline cell is the resumed process only (1189.7 seconds). The killed prefix, from before the memory fix, took about 10 minutes and isn't in that cell. The moving averages and the 500-episode means are recomputed from `episode_rewards.csv` and `eval_sampled.csv`, and the entropy is the last row of `curve.csv`.

On these files the lr 0.001 policies score higher than both primary runs, in the training moving average and in the 500-episode evaluation. The 2000-episode sweep had the opposite order, because lr 0.001 was still near −17 when the sweep stopped. The longer runs also trained about ten times longer (3.00 hours against 0.27 and 0.33), so learning rate and training length are mixed together. I chose lr 0.001 after seeing the lr 0.01 results.

The baseline's lead at lr 0.001 is +16.26 against +15.56 on the training MA100, which is under one point, from one seed each. The evaluation gap is about one point (16.77 against 15.65). That is no evidence either way on whether the baseline helped.

run	episodes	training MA100	500-ep sampled	greedy
P0, no baseline	5232	−2.08 (peak +2.65)	−2.62 ± 7.24	−2.59 ± 7.48
P1, value baseline	7488	+10.65 (peak +12.68)	+10.50 ± 4.75	+10.56 ± 4.76
P2, EMA baseline	6016	+4.10 (peak +7.61)	+5.19 ± 7.46	+5.01 ± 6.89

On this seed, at lr 0.01, the value baseline is worth about 13 points over no baseline, and the EMA sits between them. The value-baseline evaluation matches its training moving average. It is one seed, and the CartPole matrix shows how much a single seed can flatter either method. Pong got one seed because of the wall-clock limit, and that limit is what this table reflects.

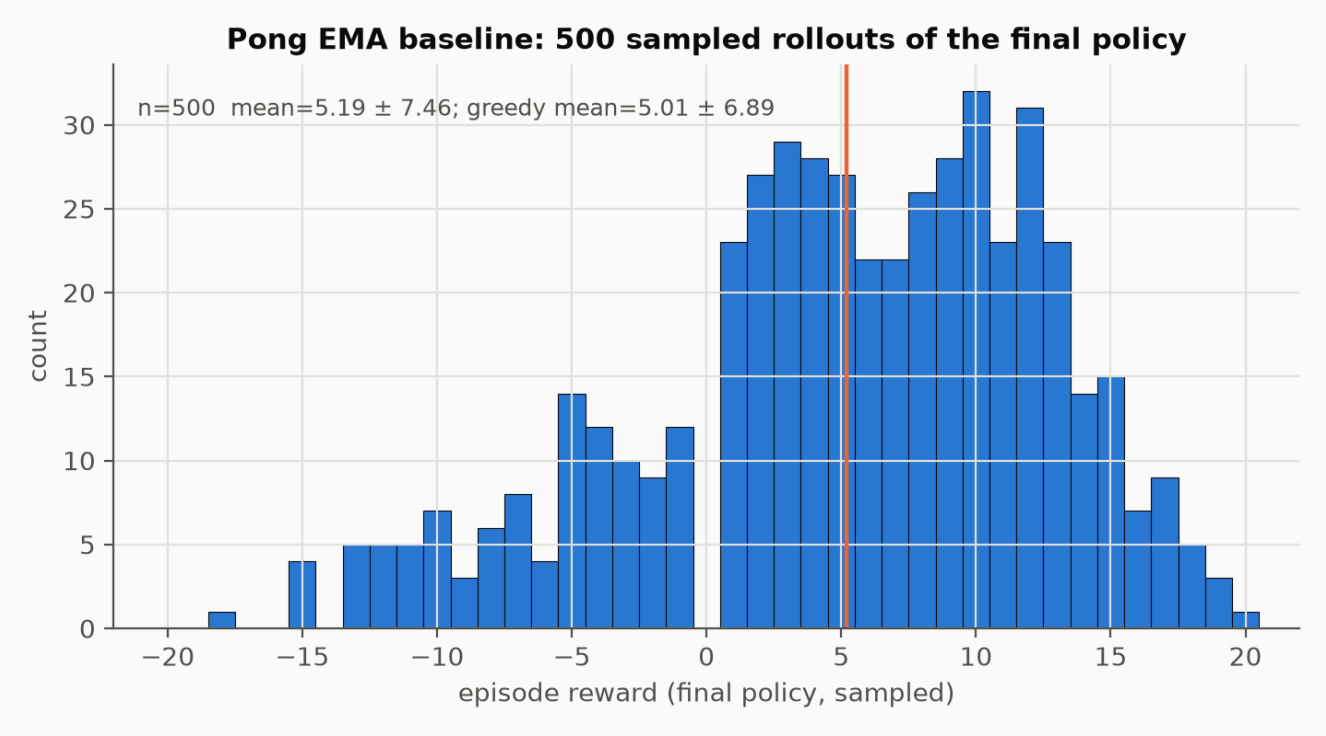


Figure 20. 500 sampled rollouts of the EMA policy. Mean 5.19 ± 7.46, n = 500, range −18 to +20. The distribution is wide and has weight on both sides of zero, unlike the value-baseline histogram, which is mostly above zero.

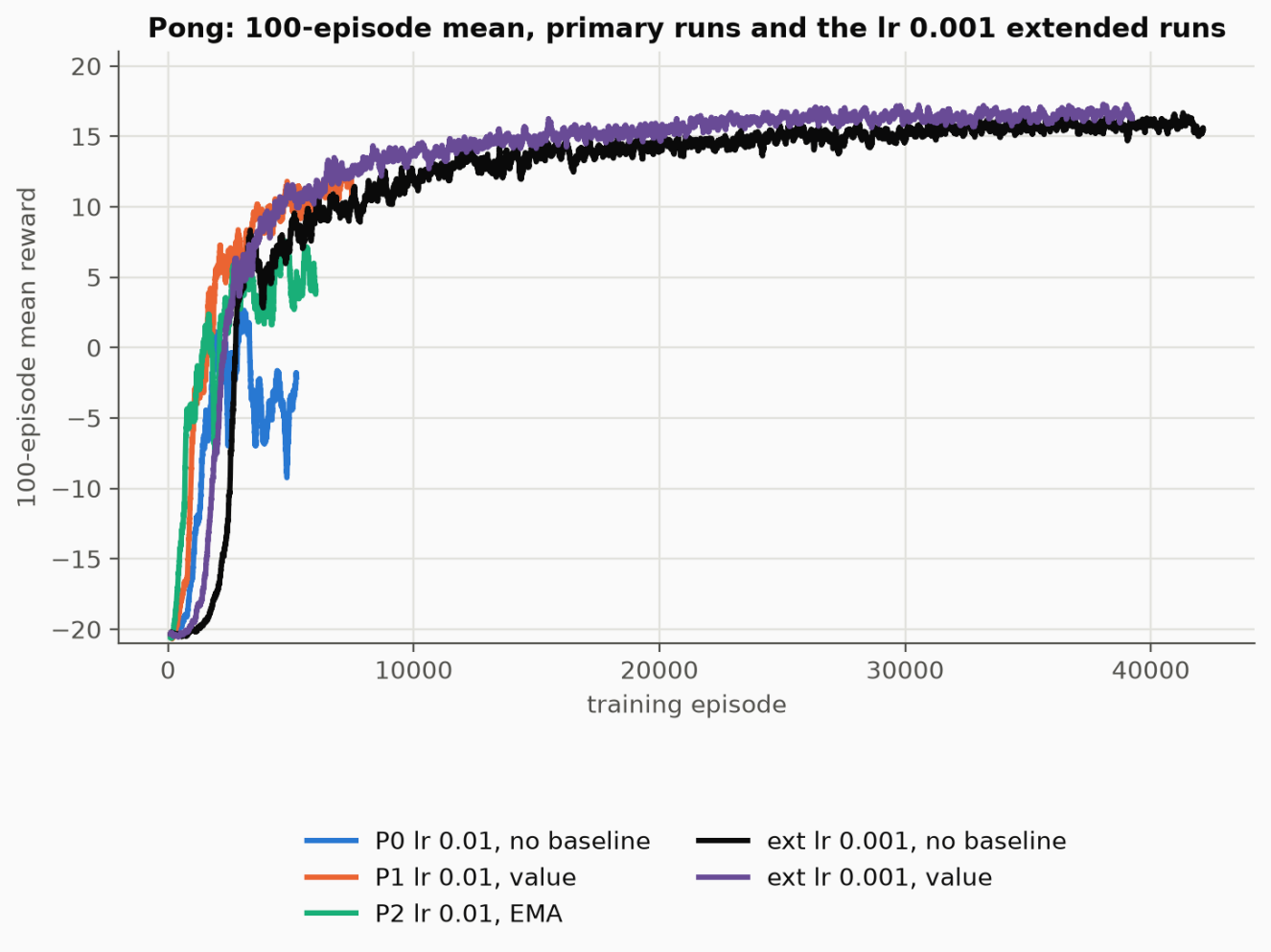


Figure 21. 100-episode mean of every Pong run. The lr 0.01 runs stop within the first few thousand episodes: no baseline near −2, EMA near +4, value baseline near +11. The two lr 0.001 runs continue to about 40,000 episodes and finish near +16. They also trained for 3 hours rather than about 16 to 20 minutes, so the picture mixes learning rate and training length.

The EMA late checkpoints, 50 episodes each, scored 7.7, 4.6, 6.5, 8.9 and 6.2. That band is wider than for Part 2 or the value baseline (which stayed within about one point), but it's nothing like the CartPole spread of more than 100. P2 stopped at episode 6016 because the moving average had moved from 4.11 to 4.10 over 3000 episodes.

Edge cases across the three parts

what can go wrong	handling	test	measured
Cap makes customers free	solved caps 10, 30, 200, 300, 400	`cap_10_artifact`	cap 10: $V(0)=356$, never dispatch; larger caps agree
Holding-cost timing	solved both timings	`cost_timing_sensitivity`	threshold 13 either way; $\Delta V(0)=120$
Value assumed convex	reported the failure	`V_convexity_observation`	min second difference -3.78, not held
Return accumulated forward	backward loop, hand values	T-R1	`[1.75, 1.5, 1.0]`, not `[1, 1.5, 1.75]`
`w_t=1` treated as the discounted gradient	enumerated both estimators	T-R2	unweighted differs by 147 SE
Forgotten `zero_grad`	test requires that omitting it changes weights	T-C4	0.80 vs 0.60
500-step cap treated as a true terminal	thresholds removed	T-C1	length 500, `truncated`, cause `time-limit`
Reward sign flipped by accident	train on `-r`	T-C6 / C5	last-50 length 9.2–9.4, under 15
`γ^t` erases late CartPole steps	C3 ablation	C3	MA100 ≈ 9.3 on three seeds
`γ = 0` expected gradient is zero	3 seeds × 1000 episodes	T-C8	last-100 length 9.3, 9.5, 9.4, not held
Pong preprocess mutates the frame	copy first	T-P3	input unchanged
Network emits NOOP or FIRE	index map `{0,1} → {2,3}`	T-P1	1000 steps, actions ⊆ {2,3}
Actors drift off the learner's weights	recompute log π	T-P6	max gap 0
Later points leak into earlier rallies	reset `G` at `r ≠ 0`	T-P7	`G_t = r_t` at all 21 scoring steps
Baseline uses the current episode	EMA reads previous batches only; `V` is detached	T-B3, T-B4	value grad stays empty; EMA sequence `[0, 10, 11]`
Final checkpoint is a lucky save	5 late checkpoints × 50 episodes	T-C9	seed 0 spread 60 to 267
Adam hides a baseline's smaller gradient	report variance, not step size	T-R4, T-B5	steps are ±0.01; variance ratio 0.01–0.19

The complete machine-written tables are `shuttle/validation_results.md` and `validation_pg_results.md`.

Where I departed from the defaults

- The estimator uses `w_t = 1`, not the exact `γ^t` weight. The `γ^t` version is the C3 ablation.

- Pong returns reset at every point.
- CartPole uses one episode per update, as specified. Pong uses batches of 16 episodes, with an MLP.
- The primary Pong runs use lr 0.01, as specified, and stop when the moving average is flat over 3000 episodes. The extended runs use lr 0.001, have no early stop and are capped at 3 hours. I picked lr 0.001 after seeing the lr 0.01 results.
- The CartPole primary result is the lr 0.01, one-episode-per-update run, even though its five-seed average MA100 is 88. C7 and C8 are labeled variants.
- I evaluate the final policy by sampling from it, and report greedy rollouts and a late-checkpoint spread next to it.
- The value-baseline fit in T-B5 runs for thousands of Adam epochs, because a few dozen can't leave the initialization.

Given an additional three months

I'd spend the time on checks that could still change my mind, not on more of the same plots.

On CartPole I'd run many more seeds and put a confidence interval, with a paired test, on the difference between no baseline and the value baseline. Five seeds already show that both methods sometimes collapse, and thirty would show whether the collapse rate actually differs. The `γ = 0` run collapsed, which the expectation didn't predict, so I'd repeat it at a much smaller learning rate or with many episodes per update, where the mean-zero gradient might stay near the random policy. The sweep over `γ ∈ {0.5, 0.8, 0.99}` is still missing, and I wouldn't assume a longer horizon learns faster.

On Pong, the 2000-episode learning-rate sweep was the wrong horizon. lr 0.001 looked stuck near −17 and then, over 3 hours with the early stop removed, passed both lr 0.01 policies. That comparison still mixes learning rate with training length, and it's one seed. Next I'd run the same four cells at a matched wall clock, with more than one seed. A convolutional network against this MLP, and a stronger estimator (GAE, then A2C or PPO) at the same clock, are still open. The point reset is a modeling choice, so I'd train one run without it. I'd also evaluate with sticky actions on and off, because training uses sticky actions at 0.25 and a single evaluation seed can flatter a peaked policy.

The cleanest new test is to run the same policy gradient on the shuttle MDP. Part 1 already gives the optimal value and the optimal threshold, so it's an exactly solvable testbed: the learned dispatch threshold can be compared with 13 and the learned value with 576.67, with the stochasticity reduced until the comparison is sharp. I'd also change the arrival distribution (away from uniform on 1 to 5) and the cap semantics, and see how far the threshold moves before the "cap 10 never dispatches" artifact appears.

For the baseline itself I'd estimate the optimal state-dependent baseline instead of using a neural fit stopped after a few thousand Adam steps. I'd also repeat T-B5 with enough episodes that the cosine at the high-variance checkpoint settles down. The episode-1000 cosine of 0.33 is the number I trust least.

Appendix

Preprocessing

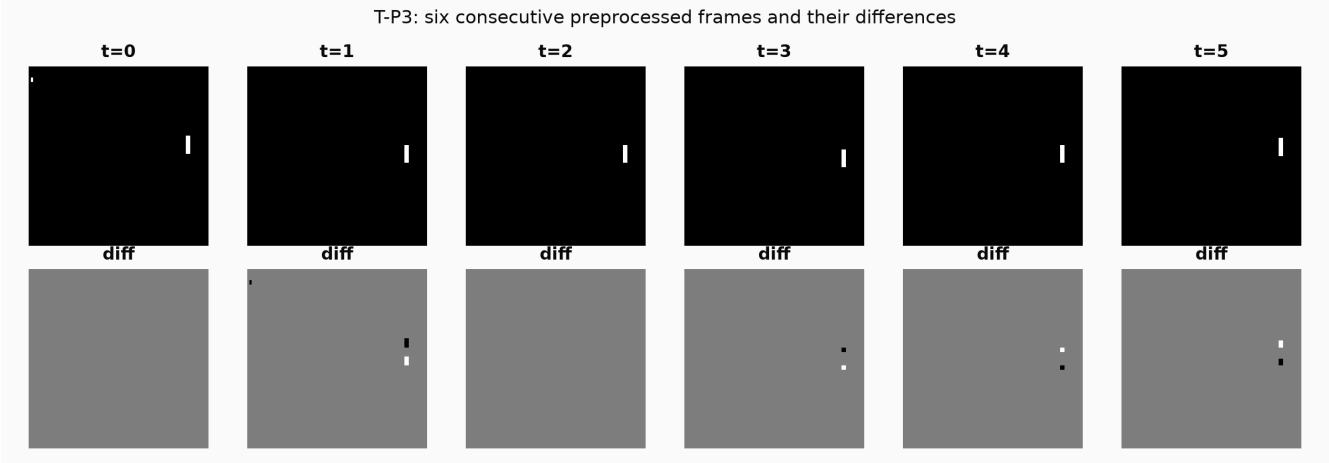


Figure 22. Top: six consecutive preprocessed frames. The right paddle is the bright vertical mark, and the ball is one or two pixels, easy to miss in a still. Bottom: the frame difference. The first difference is identically zero (no previous frame). Later differences show the paddle and the ball moving, with values in `{−1, 0, 1}`.

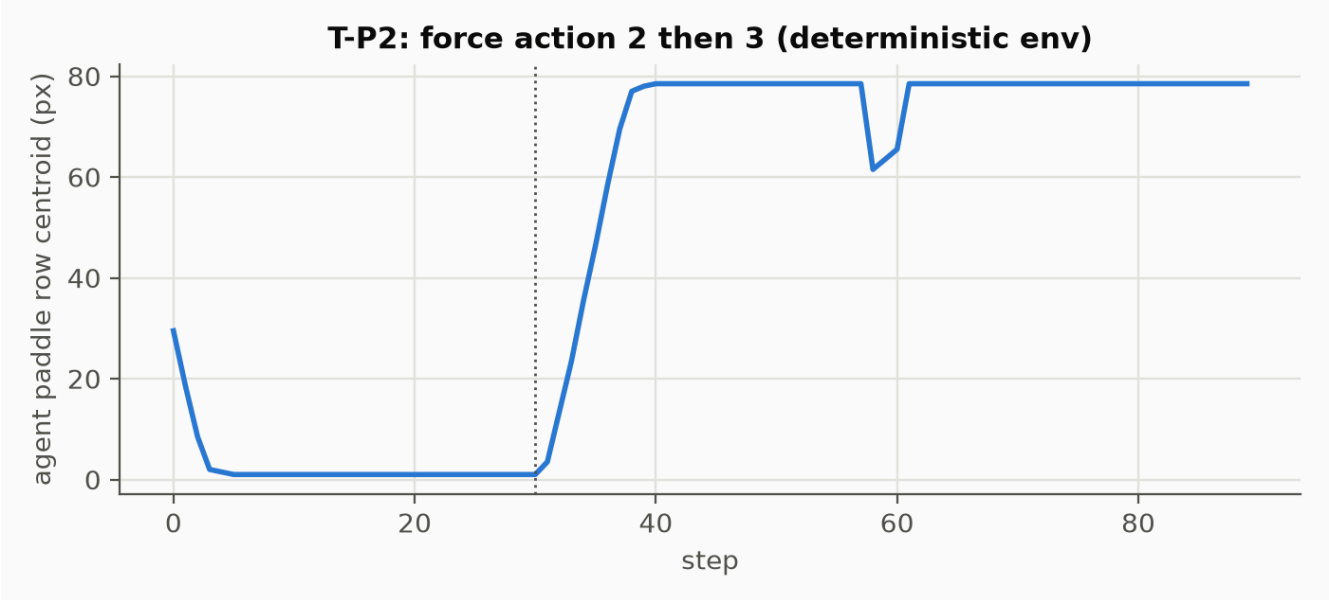


Figure 23. Deterministic environment, action 2 for 30 steps and then action 3 for 60. The right-paddle centroid travels tens of pixels and reverses. Action 2 decreases the row and action 3 increases it. The scripted controller in T-P9 uses that sign.

Throughput

Random-policy benchmark on deepdish (`results/pong_bench.json`):

workers	env steps/s	episodes/s
1	1826	1.66
8	5234	4.75
16	9497	8.84
24	12726	12.08
32	14372	13.70

At 16 workers a random episode runs at about 9 per second, so 2000 episodes take a few minutes while rallies stay short. Once lr 0.01 lengthens the rallies to about 3000 steps, the same 16 actors manage about 5 episodes per second. A 4-hour budget at 32 workers would cover about 197,000 random episodes, and the runs were capped at 25,000 regardless.